面向法规领域 RAG 系统切片审核修改可视化工作台

软件设计说明书

版本号：V1.0 著作权人：武弘正
开发完成日期：2026 年 11 月 5 日

第 1 章 概述

1.0 软件开发历程

由于身边亲朋好友多从事企业安全服务行业，在工作过程中，对于企业安全服务资料的制作是时间成本最高的部分。制作一家企业的全套相关文件，需要结合企业相关事实，以及最重要的、涉及多层次多维度不同标准的有关法律法规。面对成百上千甚至可能互相矛盾的条款，制作资料的难度可想而知，工作人员的负荷可想而知。于是，一个基于 AI 时代的企业资料制作器的雏形在笔者脑中出现了。

由于笔者能力有限，目前本科就读，在经历了短暂的 AI 相关学习后，意识到：以 RAG 检索法规以及企业事实为上游内容，结合下游相关的模板企业资料生成器，是目前可以想到的最优解。在制作此软件之前，其产出的下游使用者 RAG 系统也已搭建完毕。作为一个 DEMO，它是成功的；但作为一个工具，它是失败的——失败的不是它的召回算法本身，而是上游的切块质量。查阅相关资料后发现，切块这一个最简单的部分，反而是最复杂的部分，最熬人、最难解决问题的部分。在尝试过机器按规则切片、让 AI 去切片后，在不去训练模型的前提下，最好的解决方法就是机械前置拆分切块，然后由人来进行调整。

正如上文所说，最终由人来进行检查。但是，切块产物的可读性对人来说近乎为零：不妨设想一下，当你把《红楼梦》的第一章正文部分舍弃段落划分，全部展开铺成一行，然后要求你一字不差地读完，会是一种什么感觉？更何况法规文件中还有 HTML 表格，把一个带有表格信息的内容也像这样展开，这几乎是不可能读取的。也就是说，对于未经过任何处理的切片原件，人工处理的效率几乎为零。

在进行调查后，笔者发现很多人工切片选择使用表格来进行人工切片。这样确实可以解决问题，但在我看来，这只是把《红楼梦》从一行变成了五六行的区别，甚至无法实现复核，出现错误之后，修改的难度也可想而知。

因此，为了提高切片的质量，实现切片可视化，并且最好能够在原文中显示出切片对应的原文的哪里，并且可以实现即时的反馈，便是提高人力效率、从而提高切片质量的最优解。

1.1 编写目的

本说明书描述面向法规领域 RAG 系统切片审核修改可视化工作台（版本号 V1.0）的设计思路与实现方案，内容包括软件定位、运行环境、安装启动方式、总体架构、数据契约、模块划分、关键算法、界面交互、操作流程、异常处理、测试验收与版本维护安排。编写本说明书的目的有三个：一是作为软件著作权登记时提交的文档鉴别材料，使审查者能够在阅读文本后了解该软件的功能构成与技术方案；二是作为开发与维护的技术底稿，使后续维护人员不必重新逆向阅读全部源码即可理解模块边界与设计取舍；三是作为使用与验收的依据，使切片审核人员在遇到界面提示或错误信息时，能够查到对应的处理规则。

本说明书只描述截至开发完成日期（2026 年 11 月 5 日）已经实现并经过验证的功能。对于已经形成方案但尚未编码的后续阶段（参数化重切、渲染视图、切块器自带行号来源），本说明书不作功能承诺，只在第 12 章作为扩展点列出，并说明当前版本采用的替代做法。凡是文中出现的数量、比例、耗时等数字，均来自本机的单元测试记录与真机回归记录，未作四舍五入以外的修饰。

本软件是独立作品：它只读取下游 RAG 系统上游切块工程的产物（切片 JSONL、侧车、报告），不包含该系统的代码与数据，也不向其目录写入任何内容；著作权仅涉及本软件自身的源程序与文档。

1.2 软件定位

法规领域的检索增强生成系统通常由三个环节构成：文档解析与切片、向量化与索引、检索与生

成。切片位于最上游，切片的边界是否落在条文、款、项的正确位置上，直接决定后续检索能否命中完整的规范语义。本项目所服务的上游切块程序按照条文结构、表格结构与版面特征自动产出切片，但自动结果存在三个现实问题：第一，切得对不对看不见，切片是一行行 JSON，人工无法快速判断某一条切片对应原文的哪几行；第二，即使看出了问题也改不了，切块程序是批处理工具，不接受逐条微调；第三，改完之后存不住、也说不清改了什么，缺少可回退、可审计的落盘机制。

本软件正是为解决这三个问题而设计的桌面级工作台。它把自动切块结果与 Markdown 原文并置在三栏界面中，左侧是文件清单，中间是带行号的原文，右侧是切片卡片；任意一侧的悬停、点击、拖选都会联动另一侧，切片覆盖的原文行以彩色范围条与行底色标出，对不上的切片如实标注为未对齐而不伪装高亮。在此基础上，软件提供拆分、合并、删除、修改、新增、扩展、并入等原子编辑命令，每条命令都可撤销重做；编辑结果先以草稿形式自动落盘，再由使用者显式提交，提交时校验基线、备份旧版、原子替换正式切片文件，从而保证基线永远干净、修改永远可追溯。最终结果以 15 个契约字段的 JSONL 导出，可被既有索引流程直接读取。

需要明确本软件的边界。它不实现切块算法本身：自动切块由随包分发的切块引擎以子进程方式调用，本软件只负责调度、展示与人工修订。它不做向量化、不建索引、不提供检索服务，也不训练或微调任何模型。它不接入外部服务，不设账号体系，不上传任何数据，全部数据存放在本机目录中服务只监听回环地址 127.0.0.1。它不支持多人同时编辑同一份文档，同一时刻只应有一个使用者在工作台上操作。

1.3 适用读者

本说明书面向四类读者。软件著作权登记的审查人员可重点阅读第 1 章、第 4 章与第 12 章，以了解软件的构成、规模与开发过程；第 5 章至第 8 章提供了功能真实性的技术细节，可作为佐证材料。负责后续维护的开发人员应通读第 4 章至第 7 章，其中第 6 章按模块给出职责与关键函数，第 7 章给出算法步骤与失败模式，二者配合源码阅读效率最高。承担切片标注与审核的业务人员可只读第 8 章与第 9 章，前者说明界面元素与反馈含义，后者按步骤说明从启动到提交的完整操作路径，遇到异常时查阅第 10 章。下游检索系统的集成人员应阅读第 5 章，重点确认 15 个契约字段的含义派生字段的计算方式与导出文件的组织格式。

1.4 术语与缩略语

本说明书使用如下术语。为避免歧义，正文中「切片」与「块」同义，均指一条 chunk 记录；「原文」指 data/source 目录下的 Markdown 源文件；「基线」指未经人工修改的自动切块产物。

术语	英文或缩写	含义
切片	chunk	一条可独立检索的文本单元，对应契约中的 15 个字段组成的一行记录
切块	chunking	把长文档按结构切分为切片的过程，本软件中由外部引擎完成
契约字段	contract fields	上下游共同约定的 15 个字段，任何一方不得擅自增删
侧车	extras	与切片一一对应、按 id 关联的附加属性文件，不进入 15 字段
基线	baseline	未经人工编辑的切块产物，即 data/chunks 下的正式文件
草稿	draft	人工编辑后尚未提交的整份快照，存放于 data/draft
提交	commit	校验基线后把草稿原子写入正式产物的动作
对齐	align	判断某条切片对应原文哪几行的过程及其结果状态
行区间	span	切片覆盖的原文行范围，采用 1 起闭区间表示
工作行	work line	超长单行表格被展开后的行，是对齐搜索使用的内部坐标
分片	part	同一逻辑块因超长被均分后的第 i 段，编号形如 -p1
质检	gate	切块引擎对产物执行的规则校验，结果写入报告文件
检索增强生成	RAG	Retrieval-Augmented Generation，先检索后生成的问答范式
架构决策记录	ADR	Architecture Decision Record，记录设计取舍及其理由
行式 JSON 文件	JSONL	每行一个 JSON 对象的文本格式，便于流式读写与逐行比对

1.5 登记信息与文档约定

本软件的登记信息如下表。表中各项与软件启动后控制台打印的信息、安装包属性以及本文档正文中的口径保持一致。

项目	内容
软件全称	面向法规领域RAG系统切片审核修改可视化工作台
版本号	V1.0
著作权人	武弘正
开发方式	独立开发
开发完成日期	2026-11-05
发表状态	未发表
开发语言	Python 3（后端）、JavaScript（前端）、HTML 与 CSS（界面）
运行平台	Windows 10 / Windows 11 64 位
程序规模	后端 Python 约 4 100 行，前端 JavaScript 约 3 800 行，样式与页面约 1 000 行，测试代码约 2 700 行

本文档的书写约定如下：章标题使用「第 N 章 名称」，节标题使用两级编号；正文用完整句子陈述，表格只用于字段、参数、接口与测试用例的清单；命令行示例最多三行，正文不贴大段代码；界面元素名称与源码中的按钮文字一致，便于对照。文中提到的路径均相对于软件根目录，路径分隔符按 Windows 习惯写作反斜杠。

本软件的仓库根目录下同时保留了若干设计与过程文档，包括架构说明、交互说明、参数说明、数据契约、对齐说明、路线图、决策记录、参考清单、使用说明与打包说明。这些文档在开发过程中随代码同步更新，本说明书在其基础上按登记材料的要求重新组织，删去了过程性的讨论与尚未落地的设想，只保留与已实现功能对应的内容。

1.6 本说明书的组织结构

第 1 章说明编写目的、软件定位、读者对象与术语。第 2 章列出硬件、操作系统、运行时、浏览器与数据位置等运行环境要求，并说明目录约定。第 3 章给出三条安装路径：从源码运行、用打包脚本生成免安装目录版、用安装包脚本生成安装程序，同时说明首次启动的行为、端口选择与配置项含义。第 4 章描述系统总体设计，包括五层架构、模块职责、一次请求的完整流程、目录结构与九条关键设计约束，这一章是理解后续各章的前提。

第 5 章逐字段说明数据契约，包括 15 个契约字段的类型与语义、编号规则、字段的三个归属类别、extras 侧车的键集合、单文件记录与质检报告的形态，以及 HTTP 接口清单与统一的错误返回约定。第 6 章按模块说明职责边界与关键函数，覆盖服务入口、路由层、领域门面、编辑命令、对齐、自动分片、切块调度、草稿层、清单层、配置层、路径层与引擎适配层。第 7 章展开关键算法，包括原文与切片的三层对齐、相似度兜底、行级与字符级拆分、合并与扩展、按行装箱的自动分片、草稿与基线的校验提交。

第 8 章描述界面与交互，按三栏布局、左侧文件树与范围条、中间原文窗格、右侧切片卡片、右键工作栏、拆分光标气泡、表格可视化与快捷键的顺序展开。第 9 章以操作步骤的形式串起完整流程从启动、选文件、看联动，到拆并删改增、草稿、提交、放弃、导出与重新切块。第 10 章列举异常与边界情形及其处理方式。第 11 章说明测试范围、单元测试分布、真机回归项与验收口径。第 12 章说明阶段划分、已完成内容、升级与备份策略以及后续扩展点。

各章之间存在引用关系：第 5 章的字段定义是第 7 章算法的输入前提，第 6 章的模块划分决定了第 7 章算法所在的位置，第 8 章的界面元素在第 9 章的操作步骤中被逐一点到，第 10 章的异常项与第 11 章的测试用例一一对应。为便于对照，同一概念在不同章节使用同一名称，例如「草稿」「基线」「提交」在第 6 章、第 9 章与第 10 章中含义完全一致，不作同义替换。

第 2 章 运行环境

2.1 硬件要求

本软件处理的对象是纯文本的 Markdown 文件，不做页面渲染、不做图像识别、不做模型推理，因此硬件门槛很低。最低配置为双核处理器、2 GB 内存、200 MB 可用磁盘空间以及 1280×720 的显示分辨率。推荐配置为四核处理器、8 GB 内存、1 GB 可用磁盘空间以及 1920×1080 的显示分辨率。这里的磁盘空间分为两部分：程序本体占一部分，数据目录占另一部分。以本机打包产物为例，免安装目录版解压后约 20.8 MB，压缩包约 9.4 MB，安装包约 9.1 MB，其中包含了 Python 解释器与切

块引擎，因此使用者无需另行安装 Python。数据目录的大小取决于语料规模，本机用于验证的法规语料共 97 份 Markdown 文件，源文件与切块产物合计约二十余兆字节。

显示分辨率之所以列入要求，是因为界面采用三栏布局：左侧文件树约 260 像素，右侧切片栏在 360 至 480 像素之间按可用宽度伸缩，中间原文栏自适应。窗口宽度收窄到 1180 像素以下时，左栏降到 250 像素、右栏降到 320 至 380 像素；收窄到 900 像素以下时，三栏改为上下两段，右栏移到原文下方并占窗口高度的四成，顶栏同时换行。在 1280 像素宽的屏幕上三栏仍可并排显示，但原文的横向余量较小，超长表格行需要横向滚动；在 1920 像素宽的屏幕上，原文与卡片都能获得较舒适的阅读宽度。界面不依赖显卡加速，集成显卡即可，浏览器缩放比例建议保持 100%，缩放后行高与表格行的坐标映射仍然按实际像素计算，不会失准。

2.2 操作系统

本软件的目标操作系统为 Windows 10 与 Windows 11 的 64 位版本，这是打包脚本与安装脚本支持的唯一平台。开发与自测环境为 Windows 11 64 位，其上运行 Python 3.12.10。由于后端只使用 Python 标准库、前端只使用原生 JavaScript，程序逻辑本身不依赖 Windows 专有接口，在 Linux 与 macOS 上以源码方式运行理论上可行；但启动脚本 run_studio.cmd 是 Windows 批处理文件，打包脚本使用 PowerShell 与 PyInstaller，安装脚本使用 Inno Setup，这三者都不跨平台，故本版本只对 Windows 作正式支持与验收。

对系统区域设置没有特殊要求。程序内部一律以 UTF-8 读写文本文件，启动脚本会先把控制台代码页切到 65001 并设置 PYTHONUTF8，因此简体中文、繁体中文与英文系统都能正确显示中文路径与中文内容。需要注意的是，批处理文件本身必须是纯 ASCII 且使用 CRLF 换行，因为 cmd.exe 按系统 OEM 代码页解析批处理，若在批处理里写入中文会导致解析错乱。项目内的启动脚本因此只包含英文提示，这一约束在脚本头部注释中有明确说明。

2.3 运行时

以源码方式运行时需要 Python 3.10 或更高版本，实测版本为 3.12.10。选择 3.10 作为下限的原因是代码中使用了联合类型写法与结构化模式匹配之外的若干新语法，且标准库 http.server 的 ThreadingHTTPServer 在该版本上行为稳定。程序不使用任何第三方包，安装依赖这一步可以完全省略，这也是「双击就能跑」这一设计目标的前提。启动脚本会先尝试 PATH 中的 python，失败后再尝试 Windows 自带的 py -3 启动器，两者都找不到时打印提示并暂停，提示内容包含 Python 官方下载地址与「安装时勾选 Add python.exe to PATH」的提醒。

以打包方式运行时不需要安装 Python。打包脚本用 PyInstaller 把解释器、标准库、本程序代码前端静态文件与切块引擎一并收进产物，生成一个可执行文件加一个 _internal 目录。可执行文件在启动时以冻结态运行，此时判断引擎调用的方式与源码运行不同：源码态执行 python -m chunking_v2.cli，冻结态则执行自身并附加隐藏参数 --run-chunk-cli，由内置的转发入口把参数交给打包进来的引擎模块。这样处理的原因是冻结态下 sys.executable 指向本程序自身，若仍按模块名调用会递归启动自己。

切块引擎是一份随包分发的第三方引擎副本，位于 vendor/splitchunk 目录，约 0.7 MB，包含引擎包与画像数据。程序不 import 引擎，只以子进程方式调用其命令行入口并读取它写出的三个文件因此引擎的版本升级与工作台代码互不影响。引擎的版本号在运行时读取其包内定义的 __version__ 字段，源码态为 0.1.4。

2.4 浏览器

界面在浏览器中运行，推荐使用 Microsoft Edge 或 Google Chrome 的较新版本，二者均为 Chromium 内核。界面使用了原生 dialog 元素、fetch、CSS Grid 与 CSS 变量，不使用任何前端框架、不使用构建工具、不加载任何外部资源，因此断网状态下功能完整。Internet Explorer 不受支持，原因是其缺少 dialog 元素与部分现代 CSS 特性。Firefox 在近期版本中可以使用，但本项目未对其做逐项回归，故不作推荐。

浏览器的本地存储只用于记住一个界面偏好，即左侧范围条是否显示，键名与值都很小，不含任何业务数据。除此之外界面不写 Cookie、不写 IndexedDB、不使用 Service Worker，因此不存在版

本升级后缓存陈旧的问题；若遇到样式或脚本异常，按 Ctrl+F5 强制刷新即可重新拉取静态文件。静态资源请求带版本查询串，页面内的脚本与样式引用也带版本号，便于在开发期区分新旧文件。

2.5 目录与数据位置

软件采用「一个配置文件加一个数据目录」的组织方式。配置文件名为 config.json，源码运行时位于仓库根目录；打包运行且安装目录可写时位于可执行文件同级；安装到 Program Files 等不可写目录时，配置与数据目录回落到当前用户的 %LOCALAPPDATA%\SplitChunkAssistant 下。数据目录由配置项 data_root 指定，可以是相对路径也可以是绝对路径，其下要求至少存在 source 与 chunks 两个子目录，缺少任何一个都会在启动时给出明确提示而不是静默失败。

路径	内容	是否必需
config.json	端口、数据目录、自动分片与自动切块等配置	首次启动自动生成
data\source\	Markdown 源文件，可含子目录作为分组	必需
data\chunks\<名>\<名>.chunks.jsonl	正式切片，每行一个契约对象	必需
data\chunks\<名>\<名>.extras.jsonl	切片侧车属性，按 id 对应	可选
data\chunks\<名>\<名>.report.json	切块引擎的决策日志与质检结果	可选
data\draft\	未提交草稿，文件名内嵌基线摘要	自动创建
data\backup\	提交前的旧版备份，保留最近三份	自动创建
data\.tmp_chunk\	引擎输出的临时目录，成功后原子搬移	自动创建
vendor\splitchunk\	切块引擎副本，升级时整体替换	打包时内置

数据目录的结构体现了一条重要约定：一个文档一个文件夹，文件夹名与文档名相同，文件夹内放同名的三个文件。这样的布局使「文档」成为目录意义上的实体，草稿、备份、切块临时产物都能按文档名归拢，也避免了不同分组下同名文档互相覆盖。程序同时兼容两种旧布局：切片文件直接平铺在 chunks 目录下，以及切片文件统一命名为 chunks.jsonl。兼容逻辑在扫描时按「优先带文档名的文件」的规则去重，保证同一文档不被重复计数。

分组由子目录产生。source 下的一级子目录名会成为左栏的分组名，chunks 下对应的子目录结构必须与之一致，配对靠「同名加同相对路径」完成。当前版本的左栏只显示一个数据源分组，分组标题形如 data 或 data/子目录名，这是因为本版本已经把数据来源收敛为本地目录一种，早期从外部清单读取多个数据源的模式已被移除，相关代码仅保留给单元测试使用。

2.6 数据规模与性能边界

本软件在设计时按「全量法规语料」确定性能目标：单文档不少于一千行原文、不少于八百条切片时界面仍可流畅操作；全部语料约一万四千条切片时，打开任一文档的对齐计算不超过两秒，命中缓存后不超过零点二秒。实际验证使用的语料共 97 份法规文本、13 764 条切片，其中单份最大的是山东省自由裁量基准，原文 606 行、757 条切片，含 684 张表格；另有若干标准类文档存在整表压缩在一行的情况，单行长度可达二十三万字符。

为达成上述目标，程序在四个位置做了取舍。原文窗格与切片栏都不做虚拟滚动：一次把整份文档的行与全部切片卡片建进文档片段，再整体插入容器，行、卡片与高亮之间的对应关系因此是静态的，联动时只需按元素查找，不必处理「滚到视口再补建」带来的一致性问题；代价是首屏构建时间随行数与块数线性增长，六百余行、七百余条切片的文档在测试机上可以即时打开，行数上万、切片数千的文档则会有可感知的等待，这一边界在上一节已经说明。高亮与选中不重建节点，只对已有元素切换类名，鼠标在原文上移动时不会触发整段重排。表格网格不随文档载荷下发，而是按文件单独请求并缓存最近八份，山东省自由裁量基准的六百八十四张表合计约四百千字节；文档载荷本身也缓存最近八份，来回切换文件不重复读盘，也不重复对齐。编辑引起重绘时，滚动位置靠关闭容器的滚动锚定并显式恢复来保持，具体做法是在样式里对两个滚动容器设置 overflow-anchor: none，重建后按表格块的起始行或卡片编号把滚动位置放回去，避免浏览器在 DOM 变化时自行跳动。

2.7 网络与安全环境

软件只监听回环地址 127.0.0.1，端口默认 8790，不监听 0.0.0.0，也不发起任何出站请求。因此它不需要联网，不经过防火墙的公网规则，正常情况下 Windows 防火墙不会弹出放行提示；若企业

策略对本地监听也做拦截，只需允许本机回环即可。程序中不存在账号、口令、令牌、遥测与自动更新，不存在把源文件或切片内容发送到任何远端的行为。

安全方面的具体措施包括：路由层只接受清单中已存在的文档键，键不匹配直接返回 404，从根上避免路径穿越；上传接口对文件名做严格校验，拒绝包含斜杠、反斜杠、双点与控制字符的名字，只接受三种文本扩展名；静态文件只允许白名单内的三个名字；请求体大小上限为 64 MB，超出直接返回 413；没有任何地方使用 eval 或拼接 shell 命令；切块引擎以参数数组方式启动子进程，不经过 shell 解释。这些措施与「纯本地单机」的定位一致，也决定了本软件不适合直接暴露到局域网或公网。

第 3 章 安装与启动

3.1 三种使用方式

本软件提供三种使用方式，按使用者身份区分。开发者或需要阅读源码的审查者使用源码运行方式，只需本机装有 Python；日常使用者使用免安装目录版，解压即用，不写注册表；需要长期使用或分发给他人时使用安装包，安装后在开始菜单建立入口并登记卸载项。三种方式共用同一份程序代码与同一套数据目录约定，切换方式不需要迁移数据，只要把 data 目录整体复制到新的程序目录，或在 config.json 的 data_root 中填写原数据目录的绝对路径即可。

无论采用哪种方式，启动后的行为都是一致的：程序在回环地址上监听一个端口，用默认浏览器打开工作台页面，控制台打印版本、地址、数据目录、切块引擎状态与文件统计。软件不会把网页包装成独立的桌面窗口，界面始终在浏览器中，这样做的原因是浏览器提供了成熟的文本选择、滚动与缩放能力，而自绘窗口在长文本选择与表格坐标换算上容易出现平台差异。

3.2 从源码运行

源码运行的入口是仓库根目录下的 run_studio.cmd。双击该文件后，脚本依次做五件事：把工作目录切到脚本所在目录；把控制台代码页设为 65001 并设置 UTF-8 相关环境变量；检测可用的 Python 解释器，先试 python 再试 py -3；用解释器执行一次空语句确认能正常运行；最后以模块方式启动服务并把命令行参数原样透传。若前两步都找不到解释器，脚本打印提示并暂停，等待使用者按键后退出，退出码为 2。

也可以在命令行中直接启动，等价命令为 python -m chunk_studio.server，常用参数如下表。参数只在本次启动有效，不写入配置文件；需要长期改变端口或数据目录时，应修改 config.json 或在界面设置面板中保存。

参数	默认值	说明
--port	取配置，初始 8790	监听端口，仅绑定 127.0.0.1
--open / --no-open	默认打开浏览器	是否在启动后自动打开工作台页面
--config	程序目录下 config.json	指定另一份配置文件
--fac-root	环境变量 FAC_ROOT 或上级目录	覆盖外部切块工程的根目录，用于推导默认产物路径
--chunks-dir	由 fac-root 推导	覆盖外部产物目录，仅在兼容旧模式时使用

启动过程中的输出是有信息量的。正常启动时依次打印版本与阶段、访问地址、数据目录、切块引擎路径与版本、文件数与切片总数。引擎不可用时打印一行带感叹号的警告，说明引擎缺失或路径不对，此时界面仍可打开并浏览既有切片，只是无法自动切块新文件。产物不可用时打印另一行警告并提示可在设置中修改路径，修改保存后立即生效，不必重启。

3.3 打包为免安装目录版

打包脚本位于 packaging\build.ps1，用 PowerShell 执行，命令为 powershell -ExecutionPolicy Bypass -File packaging\build.ps1。脚本先清理 vendor\splitchunk 下的 __pycache__ 目录，避免把与本机 Python 版本绑定的字节码打进产物；再检查打包机是否装有 PyInstaller，缺失时尝试自动安装；随后调用 PyInstaller 按规格文件收集入口脚本、标准库、前端静态文件与引擎目录，输出到 dist\SplitChunkAssistant；最后把整个目录压缩为 dist\SplitChunkAssistant-win64.zip。脚本支持两个开关：-NoZip 只生成目录不压缩，-SkipInstall 跳过 PyInstaller 的安装检查，适合离线环境已备好依赖的

场景。

产物结构为一个可执行文件加一个 _internal 目录。_internal 中与本软件功能直接相关的是三部分：web 目录下的页面、脚本与样式；vendor\splitchunk 下的引擎包与画像数据；以及若干 Python 运行库。使用者双击 SplitChunkAssistant.exe 即可启动，不需要安装 Python，也不需要管理员权限。以本机产物为例，目录版解压后约 20.8 MB，压缩包约 9.4 MB。

目录版是本项目推荐的日常使用方式，原因是它启动快、不写系统目录、卸载只需删除文件夹，且子进程调用引擎时的路径关系最简单。安装包方式在这一点上略复杂：安装到 Program Files 后程序目录不可写，配置与数据需要回落到用户目录，多了一层路径判断，因此除非需要开始菜单入口与统一卸载，否则优先使用目录版。

3.4 生成安装包

安装包由 Inno Setup 6 编译生成。打包机上安装 Inno Setup 后，执行 ISCC.exe packaging\installer.iss，输出 dist\SplitChunkAssistant-Setup.exe，本机产物约 9.1 MB。安装脚本的默认行为是「按用户安装」，目标目录为 %LOCALAPPDATA%\Programs\SplitChunkAssistant，不需要管理员权限；安装向导中提供「为所有用户安装」的选项，选择后目标目录变为 C:\Program Files\SplitChunkAssistant，此时需要管理员权限。安装过程建立开始菜单快捷方式，桌面快捷方式为可选项，同时登记卸载项。

安装包不包含数据。首次启动时程序在可写位置创建 config.json 与 data 目录树，因此「安装」与「首次运行」是两个分离的动作，升级程序不会覆盖使用者的数据。卸载时执行 unins000.exe，程序文件、快捷方式与卸载项被删除，config.json 与 data 目录保留，避免误删语料。这一取舍的代价是卸载后可能残留数据目录，需要在确认不再使用后手工删除。

安装包支持静默安装，参数为 /VERYSILENT /SUPPRESSMSGBOXES /NORESTART /SP-，适合批量部署。静默安装后可用 --port 8793 --no-open 启动程序，再请求 /api/health 验证服务是否就绪，返回体中包含程序名、版本与阶段三个字段。这一条验证方式同时被写入发布前的冒烟清单。

3.5 首次启动

首次启动时若配置不存在，程序按运行形态决定配置位置：源码运行在仓库根目录生成 config.json；冻结运行先尝试可执行文件同级目录，不可写时回落到 %LOCALAPPDATA%\SplitChunkAssistant。生成的配置包含数据目录、端口、自动分片的三项参数与自动切块的四项参数取值均为默认值。随后程序确保数据目录下存在 source、chunks、draft、backup 四个子目录，已存在的不动，缺失的补建。

首次启动的控制台输出大致如下，前三行是版本、地址与数据目录，后两行是引擎与统计。若数据目录为空，文件数显示为零，界面左栏显示空态提示，提示内容指明数据目录必须包含 source 与 chunks 两个子目录。

`SplitChunkAssistant 0.3.1 已启动：http://127.0.0.1:8790/数据目录：C:\...\Split_assistant\data 切块引擎：C:\...\vendor\splitchunk（0.1.4）`

首次进入界面时，若 data\source 中已有 Markdown 文件而 data\chunks 中没有对应产物，自动切块功能会在数秒内把这些文件排队切块，左栏右上角的状态标签显示「切块中 1/1 · 文件名」，完成后显示「已切块 N 份」。若 data\source 为空，可以点左栏的「⬆ 添加文件」按钮或把文件直接拖到左栏完成导入，导入成功同样会触发切块。

3.6 端口与实例管理

默认端口为 8790。端口可以来自三个地方，优先级从高到低为命令行参数、配置文件、内置默认值。服务启动前会先向目标端口发送一次健康检查请求，若返回的程序名与本软件一致，说明该端口上已有一个实例在运行，此时程序不重复启动，而是直接打开已在运行的页面，打印提示并换端口的命令示例，然后以退出码 0 结束。这一设计避免了双击两次启动脚本时出现的端口占用错误，也让「再打开一个窗口」这种误操作变成无害操作。

若端口被其它程序占用，绑定会失败，程序打印无法监听的原因并建议改用下一个端口，退出码为 2。服务端在创建监听套接字时显式关闭了地址复用选项，原因是 Windows 下允许复用会让第二

个实例静默绑定同一端口，两个进程同时写日志与草稿，故障难以定位；关闭复用后第二个实例必然失败并给出明确提示。修改端口后需要重启进程才生效，界面上的端口输入框旁有相应说明。

3.7 配置项

配置文件是 UTF-8 编码的 JSON，字段如下表。保存配置时程序先写临时文件再替换，避免写一半断电造成配置损坏；保存后立即重新加载数据目录与引擎信息，除端口外的改动都不需要重启。

配置项	默认值	取值范围	作用
data_root	data	相对或绝对路径	数据根目录，下需含 source 与 chunks
port	8790	1–65535	监听端口，重启生效
auto_part	false	布尔	是否在编辑过程中把超长块自动拆成真实分片
max_chars	800	不小于 200	自动分片的长度阈值
min_part_chars	200	不小于 20	文本分片的最小长度，尾段过短时与上一段重排
auto_chunk	true	布尔	是否轮询源目录并自动切块
chunker_root	空	路径	引擎目录，留空时自动探测 vendor\splitchunk
chunker_poll	3	1–60	轮询间隔，单位秒
chunker_strategy	auto	auto 或 profile	传给引擎的切块策略

配置校验在保存前执行，端口必须是整数、阈值必须是正整数、路径必须能规范化，任一不满足则拒绝保存并返回具体字段的错误说明，不会出现「保存成功但一半没生效」的情况。数据目录的校验更严格：保存后若仍无法初始化数据源，接口返回错误并把原因带到界面，配置文件中保留使用者填写的值，便于修正而不是回退。

3.8 启动失败的排查顺序

启动失败主要有四类，排查顺序建议如下。第一类是找不到 Python，表现为批处理打印提示后暂停，处理方式是安装 Python 3.10 以上版本并勾选加入 PATH。第二类是配置读取失败，表现为退出码 2 且打印「配置文件无法读取」或「配置文件格式不对」，处理方式是修正 JSON 语法或删除配置文件让其重新生成。第三类是数据目录不可写，表现为提示把程序放到有写权限的位置或修改 data_root，常见于把目录版解压到 Program Files 后直接运行。第四类是端口占用，表现为提示无法监听并建议换端口，处理方式是关闭占用进程或使用 --port 指定其它端口。

浏览器没有自动打开时不影响服务，控制台会打印手工访问地址。若页面白屏，先按 Ctrl+F5 强制刷新，再查看控制台是否有红色异常：静态文件缺失会返回 404 并在控制台留下记录，数据目录不可用会返回 503 并在页面顶部显示原因。上述四类以外的异常都会在控制台打印堆栈，同时接口返回统一的错误对象，界面以提示条形式展示错误文案，便于使用者把现象与日志对应起来。

第 4 章 系统总体设计

4.1 设计出发点

本软件要解决的问题可以概括为一句话：让自动切块的产物变得可见、可改、可存。这三个词对应三组设计压力。可见要求把渲染清洗后的切片正文与原始 Markdown 行对上，而两者并不是逐字子串关系，因此需要一套对齐算法与一套如实标注的状态体系。可改要求编辑动作是原子的、可撤销的不会把数据改坏，因此需要把每个动作定义成带校验的命令，并用统一的重放机制实现撤销。可存要求修改能落盘且可回退，因此需要区分基线、草稿、备份三种文件角色，并用基线摘要做提交前的一致性校验。

另一个出发点是部署形态。使用者是法规文本的标注与审核人员，不是开发者，机器上未必有 Python 环境与构建工具，因此软件必须在零依赖、零配置的前提下双击可用。这一条直接决定了技术选型：后端用 Python 标准库的 HTTP 服务，前端用原生 JavaScript，不用框架、不用打包器、不引入数据库。选型的代价是代码量比用框架多一些，前端的状态管理与渲染要手写；收益是部署与排障极其简单，任何一台装有浏览器的 Windows 机器都能跑起来。

4.2 五层架构

程序在纵向上分为五层，依赖关系严格单向，上层可以调用下层，下层不得反向引用上层。这条约束在代码评审中被反复强调，因为一旦出现反向依赖，单元测试就必须启动 HTTP 服务，纯函数的

可测性随之丧失。

层次	名称	组成	职责
L5	表现层	web 目录下的页面、脚本、样式	三栏布局、联动高亮、编辑交互、状态提示
L4	传输层	server 模块	监听回环端口、读取请求体、写回响应、打开浏览器
L3	接口层	api 模块	路由分发、JSON 编解码、参数校验、异常到状态码的映射
L2	领域服务层	studio 模块及其下属模块	文档生命周期、编辑命令、对齐、草稿、清单、自动分片
L1	适配层	fac_bridge 模块	读取本地切块产物，屏蔽外部数据结构差异

表现层只通过 HTTP 接口与后端通信，不直接读写文件，也不做契约校验；它相信后端会拒绝非法输入，自己只做即时反馈以提高操作手感。传输层只负责搬运字节，不做业务判断，不拼路径，不解析 JSON 语义；它把关的唯一一件事是请求体大小。接口层把 URL 与请求体翻译成对领域服务的调用，把领域层抛出的异常翻译成 HTTP 状态码与错误对象，自身不包含算法。领域服务层是软件的主体，它持有配置、文档缓存、编辑命令栈与草稿状态，所有业务规则都在这一层。适配层是唯一与外部产物格式耦合的地方，它把 JSONL 读成字典列表，把侧车读成按 id 索引的字典，把源文件读成行列表；上层看到的永远是普通字典与列表，不出现外部类型。

4.3 模块划分

后端共十五个 Python 文件，按职责分为入口、接口、领域、支撑四组。下表列出主要模块及其规模与职责。表中行数为实际统计值，包含注释与空行。

模块	行数量级	主要职责	关键对外接口
server	约 230	HTTP 服务、参数解析、实例探测、打开浏览器	main、build_server
api	约 340	路由表、上传校验、错误映射、静态文件	App.handle
studio	约 640	文档门面、缓存、命令重放、草稿编排	doc_payload、apply_op、commit_draft
ops	约 780	八种编辑命令、编号重排、差异统计	apply_ops、chunk_diff
align	约 400	三层对齐、规范化、窗口定位	align_chunks、locate_text_span
autopart	约 670	自动分片物化、表格按行拆分、混合块拆分	materialize_chunks
chunker	约 550	待切扫描、子进程调用、原子发布、轮询	WatchService、scan_pending
drafts	约 180	草稿读写、基线摘要、备份轮转、提交事务	commit_draft、contract_chunks
manifest	约 530	目录清单、文档键与分组、统计	DirManifest
config	约 260	配置模型、校验、合并、存取	Config.load、Config.merged
paths	约 130	运行形态判定、资源根与数据根、默认配置	resource_root、app_root
fac_bridge	约 150	产物读取与字段归一	FacBridge.chunks、load_doc
source	约 130	源文件定位、行读取、显示裁剪、行类型	read_lines、display_text
launcher	约 60	冻结态引擎转发入口	run_chunk_cli
init	约 30	版本号与阶段标识	__version__、__phase__

前端的组织方式与后端不同，它没有模块系统，全部逻辑在一个立即执行函数内，按职责分段：状态对象、元素引用、格式化与工具、左栏渲染、原文渲染、卡片渲染、联动与选中、编辑命令与弹层、草稿与提交流程、表格行选择、事件绑定与初始化。样式文件按布局、顶栏、侧栏、原文、卡片弹层、表格、状态色分组。这种「一个大文件加若干分段」的写法对框架使用者来说显得原始，但它换来的是零构建、可直接在浏览器调试、改动后刷新即生效，符合本项目的部署目标。

4.4 一次请求的完整流程

以「打开一份文档并执行一次拆分」为例，请求链路如下。第一步，页面加载后请求文件清单接口，接口读目录清单并返回分组、文件、切片数、质检状态与草稿标记，页面渲染左栏。第二步，点击某个文件后请求文档接口，接口按文档键取清单条目，读源文件的所有行，读切块产物与侧车，调用对齐算法得到每条切片的行区间与状态，再统计行数、切片数、源字符数与对齐率，组装成载荷返回。若该文档存在未过期的草稿，载荷中的块列表改为草稿内容，并在草稿字段中标明基线摘要与保存时间。

第三步，使用者在原文中点击某行、在右键菜单中选择拆分，页面弹出拆分弹层，确认后发送编辑命令。接口把命令交给领域层，领域层先取基线文档与现有命令列表，把新命令追加成一个候选列表，从头重放一遍；重放过程中任何一条命令失败都会抛出异常，接口把它翻译成 400 并带上具体原因，此时命令不被记入列表，页面状态不变。第四步，重放成功后领域层把候选列表存下，清空重做栈，把文档标记为有未保存修改，返回重放后的完整载荷，页面据此重建卡片与高亮。

第五步，两秒防抖后页面自动请求保存草稿，接口取当前载荷中的块列表，按基线摘要写入草稿文件，同时更新内存中的草稿元信息。第六步，使用者点击提交，接口先保存一次草稿确保磁盘为最新，再重新加载基线文档，然后进入提交事务：读取正式切片文件计算摘要，与草稿文件名中的摘要比对，不一致则报「草稿已过期」并中止；一致则把正式文件按时间戳备份到备份目录并轮转，把草稿裁剪成十五个契约字段后原子替换正式文件，删除草稿文件，最后重新扫描数据目录并返回新的文档载荷。整个过程只有最后这一步会改动正式产物，之前任何一步失败都不会留下半成品。

4.5 前端结构

前端的核心是一个状态对象，字段包括文件清单、当前配置、当前选中文件、当前文档、当前选中切片、悬停切片、行到切片的索引、切片到卡片元素的索引、当前高亮的行元素、选区命中的切片集合、选区行范围、表格数据、表格块行区间、表格行索引、表格行选区状态、多选集合与草稿防抖定时器。所有渲染函数都读这个对象，所有交互都写这个对象，没有第二份真源，因此不会出现「界面显示与数据不一致」这类状态分裂问题。

渲染策略围绕三个点展开。原文窗格与卡片栏都是一次性构建：用文档片段把全部行或全部卡片建好，再整体替换容器内容，这样行元素与卡片元素的下标与数据下标一一对应，查找是常数级；代价是重绘成本与文档规模成正比，因此凡是改动数据的操作都只在提交成功后才整体重绘一次，中途的悬停、选中、拖选只改类名。联动高亮不重建 DOM，只对已有元素切换类名，且滚动请求用动画帧合并，避免一次鼠标移动触发多次滚动。行到切片的查找使用排序加前缀最大值的结构做二分，一次查找为对数级，即使一万行也只有十余次比较。

交互事件统一委托在容器上，通过元素上的数据属性识别行号或切片编号，而不是给每个元素绑定监听器。这样做的直接收益是每次整体重绘后不需要重新绑定，行与卡片的监听器数量与文档规模无关；间接收益是事件处理函数数量恒定，长时间使用不会因监听器泄漏而变慢。

4.6 目录结构

程序目录的组织方式如下表。与运行无关的开发资料放在 docs、prototype、reference、scratch 四个目录，打包时不进入产物；tests 目录只在开发期使用；vendor 与 packaging 在打包时被读取。

目录或文件	说明
run_studio.cmd	Windows 启动脚本，纯 ASCII 加 CRLF
chunk_studio\	后端包，十五个模块
web\	页面、脚本、样式三个文件
tests\	十四个测试模块与夹具
vendor\splitchunk\	切块引擎副本与来源说明
packaging\	打包脚本与安装脚本
data\	数据目录，含 source、chunks、draft、backup
docs\	十篇设计与使用文档
dist\	打包输出，不进入版本库

4.7 九条关键设计约束

第一，只处理 Markdown。法规语料在本项目中已经以 Markdown 形式存在，共 97 份，其中标准类文本多为扫描件转写，没有版面坐标。引入 PDF 解析、光学字符识别或版面模型会带来大量依赖与不确定性，收益却只覆盖少量文档，因此明确不做。这条约束决定了源文件的读取方式是按行读文本，也决定了「原文」在界面上的形态是带行号的源码行，而不是渲染后的版面。

第二，上游产物只读。本软件读取外部切块工程产出的切片、侧车与报告，但绝不改写它们的代码与数据；人工修改全部写入本软件自己的数据目录。这条约束保证了两套系统可以并行演进：上游升级切块算法不会破坏工作台，工作台的人工修改也不会污染上游的自动结果。

第三，契约不变。十五个字段是上下游共同约定的接口，本软件不新增、不删除、不改名。派生字段一律由程序重算，界面不提供手工填写入口。需要携带的附加信息放在侧车中，侧车按 id 关联不进契约。这条约束使得导出文件可以被既有索引流程直接消费，无需任何转换脚本。

第四，修改写覆盖层而不是改基线。人工编辑不直接改写正式切片文件，而是先写草稿，再由显式提交把草稿提升为新的基线。这条约束带来三个好处：误操作可以整份放弃；提交前可以比对差异每次提交都有旧版备份可回退。代价是多了一次「保存草稿」的动作，使用者需要理解草稿与提交的区别，界面用徽章与按钮状态反复提示这一点。

第五，零第三方依赖。后端只用标准库，前端不用框架，样式不用预处理器。这条约束的收益在部署与审查两个环节：部署时不需要联网安装依赖，审查时任何一行行为都能在仓库内找到来源。

第六，本地单机。只监听回环地址，不设账号，不发网络请求，不做多人协作。这条约束把「权限」「同步」「冲突合并」三类问题整体排除在外，使软件可以专注于切片本身。代价是同一份数据不能被两个人同时编辑，这一点在使用说明中被明确写出。

第七，对齐如实标注。对齐结果分为精确、包含、表格行、表题、近似、手工、未命中七种状态界面用不同颜色与符号区分，未命中的切片显示为灰色虚线并且不画行区间。这条约束的意图是防止「看起来很对」的假象：如果一段文字在原文中找不到，宁可不标，也不能标到相近的位置上让使用者误以为已经对齐。

第八，不接入外部检索平台。外部平台自带的切块器不懂法规的条、款、项结构，且需要部署一整套服务组件，与本项目的单机定位冲突。本软件只做切片与导出，检索与生成由下游系统负责。

第九，先提案后动手。这是开发流程上的约束：任何涉及行为变化的改动，先写清现状与根因、要改哪几个文件、具体改法、影响面、验证方式与风险边界，经确认后再编码。这条约束在多人协作的项目里常见，在本项目这样的单人项目中同样有价值，因为它把「想到就改」的冲动挡在了编码之前，减少了返工。

4.8 分层的收益

分层带来的第一个收益是适配层单点隔离。外部切块工程的字段命名与结构在版本间有过调整，例如侧车曾有平铺与嵌套两种写法，产物目录曾从统一命名改为按文档命名。这些差异全部收敛在适配层与清单层，领域层以上看到的是统一结构，改动不会扩散。

第二个收益是算法纯函数化。对齐、编辑命令、自动分片三个模块都不依赖 HTTP，也不打日志输入是字典与行列表，输出是新列表。单元测试可以直接构造输入断言输出，不需要起服务、不需要临时目录。目前全部 217 个测试用例在 2.8 秒内跑完，其中对齐与编辑命令两部分占了七成，这种测试密度只有在纯函数设计下才可能维持。

第三个收益是无框架前端带来的可预测性。页面没有构建步骤，浏览器加载的就是仓库里的文件调试时看到的行号与源码一致；没有虚拟 DOM 的协调过程，高亮切换的代价可以精确估算。对于本软件这种「大量静态文本加少量状态变化」的界面，手写渲染反而比框架更直接。

第四个收益是基线永远干净。因为所有修改都在草稿层，正式产物只在提交这一瞬间被整体替换所以任何时刻的正式产物都对应一次完整的、可复现的切块结果或一次完整的人工修订，不存在「改了一半」的中间状态。这一点对审计很重要：拿到的任何一份正式文件，都能从备份链中找到它的前一版，也能通过草稿机制还原修订过程。

第 5 章 数据契约设计

5.1 契约的地位

数据契约是本软件与上下游之间的唯一接口。上游是自动切块流程，下游是检索、向量化与展示系统，两侧都按十五个字段读写，本软件在中间做人工修订，但绝不在字段上做任何扩展。契约的稳定性带来两个直接结果：第一，本软件导出的文件可以被既有索引流程原样消费，不需要转换脚本；第二，上游升级切块算法时，只要契约不变，工作台不需要跟着改。因此本章把字段逐个说明清楚，并把哪些字段可以改、哪些只能由程序计算、哪些属于文件级继承这三件事分开叙述。

在设计上有一条明确的界线：契约只描述「切片是什么」，不描述「切片在界面上怎么显示」。行区间、对齐状态、是否手工修改、分片行段这些信息都属于视图层，随接口返回但从不写入契约文

件。需要跨越会话保存的附加信息放在侧车里，侧车按切片 id 关联，格式是每行一个对象，与切片文件一一对应。

5.2 十五个字段

序号	字段	类型	是否派生	含义与取值
1	id	字符串	否	切片编号，形如法规名-0001，分片追加 -p1
2	chunk	字符串	否	渲染清洗后的正文，检索与向量化只消费这一个字段
3	law_name	字符串	否	法规名，取源文件名去扩展名
4	law_num	字符串	否	条号或表号，如「第一条」「5.4.1」「表 A.3 第 1-4 行」
5	law_invalid	字符串	否	失效日期，缺省占位为 2030-12-12
6	law_type	字符串	否	法律、行政法规、部门规章、地方性法规、国家标准、行业标准
7	law_level	字符串	否	效力层级，取值随 law_type 而定
8	issuer	字符串	否	制定机关
9	effective_date	字符串	否	生效日期，允许为空
10	source	字符串	否	来源标识，取值为「法律法规」或「GB」
11	chunk_kind	字符串	否	块类型，取 clause、table、title、front、appendix、stub、paragraph 之一
12	token_len	整数	是	长度估计，等于正文字符数
13	part_index	整数	否	分片序号，从 1 起
14	part_total	整数	否	分片总数，不小于 1
15	chunk_hash	字符串	是	规范化正文的摘要前十六位，用于去重与审计

字段 12 与字段 15 是派生字段，程序在每次正文变化时重算，界面上不提供输入框，接口收到手工填写的值也会被覆盖。派生规则很直接：token_len 取正文字符数；chunk_hash 先把正文做规范化（统一换行、去掉行尾空白、压缩连续空行），再取摘要的前十六位。这样同一条正文无论经由哪条路径产生，得到的摘要都相同，去重与比对才有意义。

字段 5 的占位值需要解释。失效日期在法规文本中往往缺失，上游约定用一个远期占位日期表示「尚未失效」，本软件沿用该约定，不做推断。字段 6、7、8、9、10 属于文件级属性，新建切片时从同文件的第一条切片继承，界面的高级面板允许逐条覆盖，覆盖后与文件级默认值不一致时会提示以便使用者知道自己改动了什么。

5.3 编号规则

编号的形状由正则 ^(.+)-(\d{4,})(?:-p(\d+))?$ 描述，即法规名、连字符、至少四位的序号，可选的分片后缀。序号的来源分两种：条文类文档用条号数值，标准类与通用类文档用流水号。同一逻辑块的多个分片共享同一个基础序号，只在尾部追加 -p1、-p2，因此按编号排序即可把分片聚在一起。

编号的生成与重排有六条规则。第一，未修改的切片保留原编号，不做整体重排。第二，拆分产生的下半块取当前最大序号加一，暂不重排，这样拆分动作对既有编号的扰动最小。第三，合并保留前一块的编号，后一块的编号消失，产生编号空洞，这是可接受的。第四，集体合并保留第一块的编号。第五，新增切片按其在原文中的位置重排编号：新块取得插入点的序号，插入点之后的切片序号顺延，同组的分片共用基础序号。之所以在新增时重排而不沿用最大序号加一，是因为新增通常发生在文档中部，若给一个队尾编号，编号顺序就与原文顺序脱节，后续阅读与比对都会别扭。第六，删除分片后，同组剩余分片重新编号，只剩一个时去掉分片后缀，恢复为普通块。

重排规则与「编号是不可变身份」的直觉相反，但它与「编号表达位置」的用法一致。为了不让重排干扰差异统计，差异统计不使用编号做配对，而是用内容签名做最长公共子序列配对，详见第 7 章。

5.4 字段的三个归属类别

十五个字段按来源分为三类，这个分类决定了编辑时每个字段从哪里取值。

类别	字段	取值来源	编辑行为
文件级继承	law_name、law_type、law_level、issuer、effective_date、source、law_invalid	上游配置的组默认值与文件名	新块继承同文件首块，可在高级面板覆盖

类别	字段	取值来源	编辑行为
块级身份	id、chunk、law_num、chunk_kind、part_index、part_total	切块结果与人工编辑	可经命令修改，受校验约束
派生	token_len、chunk_hash	由正文计算	只读，每次正文变化时重算

文件级字段之所以不在每次编辑时重新推导，是因为它们的真值来自上游的分组配置，工作台没有这份配置的完整副本。继承首块的做法在实践中足够：同一份文档的所有切片本就属于同一法规，文件级字段相同；若上游对不同切片设置了不同值，例如附录部分效力不同，人工覆盖可以保留差异而不会被批量改写。

5.5 编辑对字段的影响

八种编辑命令对字段的影响可以列成一张矩阵，这张矩阵同时也是第 6 章命令实现与第 11 章测试用例的依据。

命令	编号	正文	条号	类型	分片字段	行区间与对齐
拆分	上半保原号，下半取最大序号加一	按行或按字符切分	下半可指定，留空则自动识别或继承	两半继承	重置为 1 与 1	按窗口内规范化行重新定位，标为手工
合并	保留前块	前块正文加换行加后块正文	前块为空时取后块	保留前块	不做处理	两块区间取并集，标为近似
集体合并	保留第一块	各块按文档顺序以换行连接	首块为空时依次向后找	保留第一块	不做处理	全部有区间时取并集，否则置空
扩展	保留目标块	可传新正文，否则按顺序拼接并补选区多出的源行	可传新值，否则继承	保留目标	不做处理	取选区或并集，标为手工
删除	其余不变	不涉及	不涉及	不涉及	若删的是分片则重排同组	不涉及
批量删除	其余不变	不涉及	不涉及	不涉及	一并重排	不涉及
新增	按位置重排	取命令正文	留空则按前后邻居倒推	默认段落，可指定	1 与 1	取命令中的区间，标为手工
修改	不变	可改，改后重算派生字段	可单独改	必须在七个枚举内	可改，需满足序号不大于总数	改正文则脱钩并标为手工

从矩阵里能看出两个设计倾向。第一个倾向是「能保留就保留」：合并与扩展保留前块或目标块的编号、类型与文件级字段，只在必要时才补值。第二个倾向是「改正文即脱钩」：一旦正文被人工改写，它与原文的行区间就不再可信，因此标记为手工并把区间置空或改为手工区间，避免高亮到错误的位置。条号单独修改不影响行区间，因为条号是检索字段，与正文位置无关。

5.6 extras 侧车

侧车是与切片文件同目录、同名的另一个文件，每行一个对象，按 id 与切片对应。它承载的是「有用但不属于契约」的信息，典型的是表格的结构化数据与切块引擎的判定依据。侧车有两种写法字段平铺在对象上，或写成 id 与 extras 两个键的嵌套。读取时两种都接受，写出时统一用平铺。

键	类型	含义	编辑侧处理
clause	字符串	所属条号，用于面包屑	拆分下半块与新增块继承
region	字符串	区域，取 body、appendix、front、text	继承，可高级修改
ref_title	字符串	引用标题	继承
num_cn	字符串	中文数字序号	继承
profile	字符串	切块画像名	只读，新块不继承
namespace	字符串	命名空间，缺省 body	只读，导出时补齐
probe	对象	引擎判定依据，含家族、置信度与标题模式	只读，页面只显示三个摘要字段
table_label	字符串	表号，如「表 A.3」	可改
table_title	字符串	表题	可改

键	类型	含义	编辑侧处理
table_rows	整数	表格行数	可改
table_mode	字符串	表格模式，整表或打包	可改
table_grid	对象	表格网格，含表头、行、表头行数、列数	大字段，单独接口下发

侧车的继承策略是「最小必要」：拆分下半块与新增块只继承 clause、region、ref_title、num_cn 四个键，profile、namespace、probe 不继承，因为它们描述的是自动切块当时的判定上下文，对新产生的人工块没有意义。合并保留前块的侧车，修改保留原侧车，表格块被拆分时表格信息留在上半块因为这些信息描述的是表格本身，而不是被切出来的那一半。

5.7 单文件记录与质检报告

切块引擎对每份文档产出一份决策记录，本软件只在兼容旧数据源时读取它，本地目录模式下不依赖。记录包含八个键：路径、分组、法规名、切片列表、解析结果、健全度、汇总与源字符数。健全度含切片数、可解析条号数与条号比例，条号比例在通用类与表格类文档上的阈值为零，其余为0.5。汇总含切片数、单元数、额外分片数、切片内字符数、源字符数、字符覆盖率、类型分布、缺口数、警告数、最大切片长度与超长切片数。解析结果是解析器的数据类实例，序列化时必须转成字典这是当初接入时踩过的一个坑，代码中保留了相应注释。

质检报告是另一个文件，含是否通过、切片数、失败项与指标。指标共二十六项，包括行覆盖率目录泄漏、坏块比例、引用序列、重复引用比例、表格块数量与警告数等。判定阈值有五条：行覆盖率不低于 0.60，坏块比例不高于 0.02，重复引用比例不高于 0.01，块数容差 0.40，警告数不超过5。报告中的家族字段取标准、法规、表格、通用四类之一，置信度是引擎对该判定的自评，特征字段是十六项行级特征的统计。工作台把是否通过与失败项显示在左栏文件的徽章上，通过显示对勾，不通过显示警告图标并列出失败项摘要。

5.8 接口清单

接口按资源划分，全部返回 JSON，只有页面与静态文件返回文本。下表列出当前版本实际提供的接口。表中「键」指文档键，形如 data/法规名 或 data/子目录/法规名，后端会校验其存在性。

方法与路径	作用	主要参数	返回要点
GET /api/health	健康检查与实例识别	无	程序名、版本、阶段
GET /api/files	文件清单	无	分组、文件、切片数、质检、草稿标记、配置
POST /api/reload	重新扫描数据目录	无	同文件清单
POST /api/upload	批量导入源文件	files 数组，每项含名字与内容	已创建、已跳过、错误列表
GET /api/watch	切块监控快照	无	引擎、开关、进度、待切列表、上次结果
POST /api/watch	开关自动切块	enabled	快照与提示
POST /api/chunk	手动触发切块	key 可选	是否启动、总数或提示
GET /api/config	读取配置	无	配置项、引擎信息、校验错误
POST /api/config	保存配置	配置片段	新配置与提示
GET /api/doc	单文档载荷	key	原文行、切片、对齐、统计、草稿、差异
GET /api/tables	表格网格	key	按切片 id 索引的网格与表题
POST /api/op	执行编辑命令	key 与 op	重放后的完整载荷
POST /api/undo	撤销	key	同文档载荷
POST /api/redo	重做	key	同文档载荷
POST /api/reset	丢弃会话内编辑	key	同文档载荷
POST /api/draft/save	保存草稿	key	文档载荷与提示
POST /api/draft/discard	放弃草稿	key	文档载荷与提示
POST /api/draft/commit	提交草稿	key	文档载荷、提交结果与提示

方法与路径	作用	主要参数	返回要点
GET /api/export	导出	key	文件名、切片数与文本内容
GET /	工作台页面	无	页面文本
GET /static/app.js	前端脚本	无	脚本文本
GET /static/style.css	样式	无	样式文本

接口约定有四点。第一，写操作一律用 POST，请求体为 JSON 对象，空的或非对象的请求体直接拒绝。第二，错误返回统一为包含 ok 为假与 error 文本的对象，同时带上语义化的状态码：参数或业务校验失败为 400，文档或切片不存在为 404，切块请求无法开始为 409，请求体超过上限为 413，数据源不可用为 503，未预期异常为 500 并在控制台打印堆栈。第三，读接口在数据源不可用时返回 503 而不是空列表，避免界面把「不可用」误显示为「没有文件」。第四，静态文件只允许白名单内的三个名字，其余路径一律 404，防止通过路径拼接读取任意文件。

第 6 章 核心模块设计

6.1 模块概览

后端代码按「入口、接口、领域、支撑」四组组织，共十五个文件、约四千一百行。依赖方向与第 4 章的五层架构一致：入口与接口依赖领域，领域依赖支撑，支撑之间只有少量相互引用，且都指向更基础的模块。本章逐个说明模块的职责、关键函数与设计边界，函数名与源码保持一致，便于对照阅读。

需要先说明一条贯穿全章的约定：模块对外只暴露少量函数或方法，内部实现一律以下划线开头对外接口的命名偏向业务语义，例如文档载荷、应用命令、提交草稿，而不是数据库式的增删改查。这样命名的好处是调用点读起来像在描述业务动作，缺点是函数名较长。全项目没有使用单例、依赖注入容器或事件总线，模块之间通过显式传参协作，任何一条调用链都能在十几行内读完。

6.2 服务入口 server

server 模块负责把 HTTP 协议翻译成对接口层的调用，并管理进程生命周期。核心是继承自标准库的请求处理类，它实现 GET、POST 与 OPTIONS 三个方法，三者都转发到同一个分发函数。分发函数先读请求头中的长度字段，长度非法或超过六十四兆时直接返回 413，否则读满请求体再交给接口层，最后把接口层返回的状态码、内容类型与字节串写回。响应头统一带不缓存指令，避免浏览器缓存住旧的接口结果。

服务类在这里做了一个不常见的设置：关闭地址复用。标准库默认允许复用，在 Windows 上这意味着第二个进程可以静默绑定同一端口，两个实例同时处理请求、同时写草稿，问题极难定位。关闭复用后第二个实例必然绑定失败，程序捕获异常并打印建议换端口的提示，行为可预期。

启动流程由主函数串起：解析命令行参数、加载配置、确保数据目录存在、构造领域门面、启动切块监控线程、探测端口上是否已有本软件实例、绑定监听、打印四行启动信息、按需延时打开浏览器、进入服务循环。探测实例的方式是向目标端口请求健康检查接口并核对返回的程序名，而不是只看端口是否可连接，这样才不会把别的程序误认为本软件。

打开浏览器在 Windows 上优先用系统的 startfile 调用，失败后退回标准库的 webbrowser 模块。这样处理的原因是冻结成可执行文件后，webbrowser 有时找不到默认浏览器，而 startfile 走的是系统文件关联，稳定性更好。打开动作放在后台线程里延时零点六秒执行，避免服务尚未进入循环时浏览器请求落空。

6.3 路由接口 api

api 模块集中定义全部路由，入口是一个类，构造时接收领域门面与静态目录，之后每次请求调用同一个处理方法。处理方法做三件事：解析 URL 与查询串、按方法与路径匹配路由、把异常翻译成错误响应。所有路由写在同一个方法体内，按路径顺序排列，读代码时从上到下即可看完全部接口不需要在多个装饰器与文件中跳转。

路由分四组。第一组是健康检查与文件清单，后者在数据源不可用时返回 503，而不是返回空清单。第二组是维护类接口：重新扫描、批量上传、切块监控与手动切块、配置读写。第三组是文档类接口：单文档载荷、表格网格。第四组是编辑与落盘：执行命令、撤销、重做、重置、草稿保存、放

弃、提交、导出。此外还有页面与两个静态文件的路由，静态文件名走白名单校验。

上传接口的校验值得单独说明。它先校验文件名：去空白后不能为空，不能含斜杠、反斜杠与双点，不能含控制字符，扩展名必须在三种文本扩展名之内，最终统一改名为「主干加 .md」。再校验内容：必须是合法 base64，解码后不超过十六兆，整批累计不超过六十四兆，必须能按 UTF-8 解码且去掉空白后非空。通过校验后写入同目录的临时文件再原子替换，同名文件跳过而不覆盖。返回体中分别列出已创建、已跳过与出错三项，界面据此提示「导入 N 个、跳过 M 个、失败 K 个」。

错误处理采用异常加集中捕获的写法。业务代码抛出带状态码的接口异常、命令异常、配置异常草稿异常、切块异常或清单异常，处理方法按类型映射到 400 或 404，未预期异常映射到 500 并打印堆栈。这样业务代码里不需要写返回状态码，异常信息本身就是给使用者看的文案。返回体统一为包含 ok 与 error 两个键的对象，界面只需读一个字段就能显示原因。

6.4 领域门面 studio

studio 模块是后端的中枢，它持有配置、数据源适配器、文档缓存、表格缓存、命令栈、重做栈草稿表与切块监控服务。它的方法可以分成四组：草稿相关、数据源与配置、文档载荷、编辑命令。

文档载荷的构造是全模块最核心的路径。基线文档方法先查缓存，命中则把该条目移到最近使用位置并直接返回；未命中则依次取清单条目、读切片、读侧车、定位源文件、读全部行、调用对齐，然后把每行包装成含行号、显示文本、是否截断、原始字符数与行类型的对象，把每条切片包装成含契约字段、公开侧车、行区间、对齐状态与手工标记的对象，最后统计行数、切片数、源字符数、对齐数与对齐率，连同清单摘要缓存起来。缓存容量为八，超出后按最久未用淘汰。

载荷方法在基线之上叠加三层：草稿、自动分片、会话命令。它先取基线，若有新鲜草稿则以草稿块为起点，否则以基线块为起点；若开启了自动分片则先对起点做一次物化；然后按顺序重放会话命令；最后重算统计、附加差异统计与草稿状态。这里有一个容易忽略的细节：重放永远从起点开始而不是在上一次结果上叠加，因此撤销只需要弹出最后一条命令再重放，不需要为每条命令准备逆操作。这个选择牺牲了一点性能，换来的是状态永远可以从起点推导出来，不会出现「撤销到一半状态不对」的问题。

命令方法先做校验再记账。它取出起点与现有命令，把新命令追加成候选列表，用同一个重放函数跑一遍；跑通才把候选列表写成新状态、清空重做栈、标记有未保存修改；跑不通则异常上抛，状态不变。校验与执行复用同一段代码，避免了「校验通过但执行失败」的两套逻辑不一致。

草稿相关方法在下一节与 drafts 模块一起说明。配置方法把当前配置、引擎信息、数据目录校验结果组装成载荷；保存配置时先合并再校验，校验通过才落盘并重新加载数据源，重新加载后若数据源仍不可用则报错，但已写入的配置保留，便于使用者修正路径而不是被回退覆盖。

6.5 编辑命令 ops

ops 模块实现八种命令：拆分、合并、集体合并、扩展、删除、批量删除、新增、修改。模块顶部集中定义正则与常量，包括编号正则、分片后缀正则、条号正则、层级号正则、锚点正则、七种块类型的集合、缺省失效日期、新增占位条号与继承侧车键集合。

基础函数负责字段派生与校验。文本规范化统一换行、去掉行尾空白、压缩连续空行；摘要取规范化文本的十六位；长度取字符数；派生函数在正文变化时同时更新这三个值；校验函数确认正文去空白后非空，为空则抛「正文不能为空」一类的错误。条号识别函数先试「第 X 条」的中文数字写法，再试层级号写法，两者都不匹配则返回空串，调用方据此决定是否继承或填写占位值。

编号相关有三个函数：找最大序号、生成下一个编号、整体重排。重排函数遍历块列表，为每块按其位置生成新编号，同时保持同组分片的相对顺序与总数，是新增与扩展后必须调用的一步。

八个命令函数的结构高度相似：定位目标、复制、改写字段、重算派生、校验、替换列表、必要时重排。拆分函数支持行级与字符级两种切法，行级要求切点在一与行数减一之间，字符级要求切点在一与长度减一之间，越界时给出可取范围的提示；切完后在原文窗口内用规范化行定位两半的行区间，若定位不到则保持空区间；下半块的条号按「显式填写、正文首行识别、切点所在源行识别、继承原块」的优先级决定。合并函数校验两块相邻，允许参数顺序颠倒，合并后区间取并集并标为近似两块中只要有一块没有区间，合并结果就置空区间并标为手工。扩展函数要求编号列表的第一项是目

标块，其余必须连续，正文可由调用方给出，否则按顺序拼接，区间取命令给出的选区或各块并集。删除与批量删除在删除分片时调用分片重排，保证同组编号连续。新增函数按插入位置生成编号并在插入后整体重排，条号留空时按前后邻居倒推，推不到写占位值。修改函数只改传入的字段，改正文时重算派生并标为手工。

重放函数是整个模块的对外入口。它复制输入、可选地先做一次自动分片物化、记录基线中出现过的全部编号到一个保留集合，然后逐条执行命令，每条执行后再物化一次、再把当前结果中的编号并入保留集合。保留集合的作用是防止新生成的编号复用已删除块的编号，避免撤销重做时出现编号冲突。命令失败时异常信息会带上序号与命令名，使用者能直接看出是第几步出的问题。

差异统计函数不按编号比对，而是按内容签名做最长公共子序列配对，再按条号加类型配对，最后按正文相似度兜底，输出新增、修改、删除三个数量与三组编号。这样即使新增导致编号整体顺延也不会被误报成「删了 N 条又加了 N 条」。

6.6 对齐 align

align 模块负责把切片的渲染正文映射回原文行区间，是第 7 章算法的主体。模块内的函数分为四层：文本清洗、规范化、扫描定位、分层对齐。文本清洗去掉 HTML 标签、把图片语法替换为占位标记、按规则处理 LaTeX 转义、加粗记号、页尾省略号与续行标记；规范化在清洗之后再去全角空格、去所有空白、转小写，使「第一条　为了……」与「第一条为了……」得到同一个键。

扫描函数在一段行数组中从给定位置向后找目标键，允许完全相等，也允许目标键是某行的子串（长度不小于二时才启用子串匹配），找到返回下标，找不到返回负一。表格行匹配更复杂一些：先把一行的单元格拼成规范串，拼接前去掉相邻重复单元格，因为合并单元格在解析后会产生连续重复的值；再在行数组里找连续命中的窗口。

分层对齐函数按块类型分流：表格块先走表格对齐，失败后再走表题锚定；其余块走正文行扫描游标单调推进但不绝对，找不到时允许回卷到文首再扫一次，这样可以容忍文档中存在重复表题的情况。汇总函数统计各状态的数量与已对齐数。文本区间定位函数是给编辑命令用的，它在给定的行窗口内按同样的规范化口径定位一段正文，返回闭区间，不跨出窗口，保证拆分后的两块不会跑到原文其它位置去。

6.7 自动分片 autopart

autopart 模块把「超长块」拆成真实块，是编辑过程中自动分片功能的实现。它的入口函数按顺序判断：长度未超阈值或已是分片则原样保留；文本与表格混在一起且超长则走混合块拆分；表格块走按行区间拆分；其余走文本分片。三种拆法都返回新块列表或空值，空值表示不需要拆。

文本分片先看首行是不是面包屑（含分隔符且不长于一百六十字），把面包屑摘出来；再按「总长除以阈值向上取整」算出目标段数，用总长除以段数得到每段目标长度，且不低于最小长度；然后按行装箱，单行超过目标长度时用标点硬切，切点不早于半段；装箱完成后做尾段重排，尾段短于最小长度时从上一段搬行过来，搬不动就接受现状并跳出，避免死循环；最后给每段重新加上面包屑。这样每一段都带完整的面包屑，独立检索时不会丢失上下文。

表格拆分只处理多行且超长的表格：按行装箱到阈值，行区间连续，每块写回「表 X 第 a-b 行」形式的条号，单行超长的大表不拆，因为拆开会把一行表格的列结构切断。混合块拆分按类型边界切开头的标题类行归入首段的头部，面包屑作为结构行只用来切换「当前是哪张表」而不进入正文，表格行按原始表格块归位，每块从原始块找回行区间与侧车。

物化函数遍历块列表，对每个需要拆的块调用上述三种拆法，全部处理完后若没有任何变化则返回原列表（而不是副本），调用方据此判断是否发生了物化。这个「无变化返回原对象」的细节省掉了大量不必要的复制。

6.8 切块调度 chunker

chunker 模块负责发现待切文件、调用引擎、发布产物、对外报告进度。常量包括引擎包名、命令行模块名、防抖秒数一点五、单文件超时三百秒。解析引擎目录时先看配置，再看随包的 vendor 目录，最后看相邻目录，都找不到时抛错并在界面上显示原因。

扫描函数遍历源目录下所有 Markdown 文件，跳过隐藏文件与刚写入不足防抖时间的文件，按相

对路径与文件名算出文档键，若目标目录没有切片文件则标记为新增，若有切片且有报告则比对报告记录的源摘要与当前摘要，不一致则标记为过期。报告缺失时不标记过期，这是保守做法：没有依据就不重切，避免把人工改过的切片冲掉。

引擎命令的拼装区分运行形态：源码态用解释器加模块名，冻结态用自身可执行文件加隐藏转发参数。执行函数为每个文件建临时目录、清掉上次残留、拼命令、设置环境变量（源码态把引擎目录加入模块搜索路径）、以超时限制运行子进程、解析标准输出的 JSON 结果、成功后把临时目录中的三个文件原子搬到正式目录、失败则保留临时目录并返回错误信息。原子搬移的做法是先在同一磁盘内改名，改名是原子操作，因此不会出现「切片文件写了一半」的中间态被界面读到。

监控服务是一个后台线程加一个单工作线程。主循环按配置间隔扫描，发现待切且开关打开时启动任务；任务依次处理文件、更新当前文件与进度、结束后重新加载数据源、再扫一次剩余待切、记录本次结果与摘要。对外提供快照、手动触发、等待空闲三个方法。快照内容包含引擎信息、开关、间隔、策略、是否运行、当前文件、已完成数、总数、待切数、待切前五十项与上次结果，界面用它渲染顶栏的状态标签与提示。

6.9 草稿 drafts

drafts 模块只做文件层面的事，不含业务判断。它定义契约字段元组、草稿后缀、备份保留份数提供摘要计算（取文件内容摘要的前十二位）、文件名主干生成（子目录用双下划线拼接，避免同名冲突）、草稿文件名生成与解析、按行读取、序列化、原子写入、保存、丢弃、备份、备份轮转、契约字段裁剪与提交事务。

提交事务是模块的核心，步骤严格。先确认草稿存在、文件名合法、正式文件存在；再计算正式文件当前的摘要，与草稿文件名中的摘要比对，不一致则抛「草稿已过期」；然后读取草稿内容，为空则报错；随后备份正式文件到备份目录并按时间戳命名，轮转只保留最近三份；再把草稿裁剪成十五个契约字段，原子写入正式文件；最后删除草稿文件，返回切片数、基线摘要与备份路径。这条链上任一步失败都不会留下「正式文件被改了一半」的状态，因为真正的写入只有一次原子替换。

模块中有一处注释解释了为什么草稿里保存的是界面块而不是契约块：草稿需要保留行区间与对齐状态，才能在下次打开时恢复高亮；契约裁剪只在提交那一步做。这样草稿文件比正式文件大一些但换来了编辑会话的完整恢复。

6.10 清单 manifest

manifest 模块提供两个类：一个读外部清单文件，一个读本地目录。本版本只用后者，前者保留给单元测试与兼容。目录清单在构造时扫描 chunks 目录，匹配两种命名方式（带文档名的与统一的），按文档目录名确定文档名，按相对路径确定分组，按同名同相对路径配对源文件，源文件缺失时给出警告但仍列出该文档，同时统计切片数、类型分布、字符覆盖率、侧车行数与质检结果。

文档键的构造规则是分组加斜杠加文档名，分组为空时用 data，否则用 data 加相对路径，因此键总是以 data 开头，界面与接口都用这个键定位文档。清单还提供分组视图、源文本读取、切片文件路径、侧车读取与载荷组装。侧车读取带缓存，按文件路径与修改时间判断是否失效，避免每次打开文档都重新解析几百行 JSON。

6.11 配置 config

config 模块定义一个数据类，字段覆盖数据目录、端口、自动分片三项、自动切块四项，以及两个保留字段（外部工程根目录与产物目录）。构造时做类型规范化：路径去空白并转绝对路径、端口转整数、布尔值转布尔、数值转整数，任一转换失败抛配置错误。类方法提供从字典构造、从文件加载、保存、合并补丁与校验。保存用临时文件加替换，写入带缩进与结尾换行，便于人工编辑。

合并补丁的规则值得说明：只处理非空值，未提供的键沿用旧值；另外有一条联动规则，若只改了外部工程根目录，而旧的产物目录恰好是由旧根目录推导出的默认值，则产物目录跟随变化，若使用者自定义过产物目录则保持不变。这条规则来自一次实际困扰：换了工程目录却忘了改产物目录，界面一直显示旧的切片。

6.12 路径 paths 与适配 fac_bridge

paths 模块解决一个具体问题：程序可能以源码方式运行，也可能被冻结成可执行文件；数据目录可能在程序目录旁边，也可能必须放到用户目录。它提供运行形态判定、资源根定位（冻结态取可执行文件所在目录，源码态取包的上级目录）、应用根定位（先试程序目录是否可写，不可写则用用户目录下的固定子目录）、默认配置路径、默认数据根、前端目录、引擎目录与候选引擎目录、数据目录骨架创建、默认配置生成。所有路径判断集中在这里，其它模块不自己拼路径，这样换部署形态时只需改一个文件。

fac_bridge 是唯一与外部产物结构耦合的模块。它接受一个清单对象或一个目录路径，暴露切片读取、侧车读取、整文档读取、源文本读取与统计五个方法。读取切片时逐行解析并做字段归一，保证上层拿到的一定是含十五个键的字典，缺失的键补默认值。它为每个文档缓存切片与侧车，缓存随清单重载而清空。

6.13 一次拆分的调用链

把上面的模块串起来看一次拆分：页面发出命令，server 读请求体交给 api，api 校验键与命令对象后调用领域门面的应用命令方法；门面取基线载荷与现有命令，构造候选列表，调用命令模块的重放函数；重放函数逐条执行，遇到拆分命令时定位目标块、按行或字符切分、重算派生字段、在源行窗口内定位两半区间、确定下半块条号，返回新列表；若开启了自动分片，每条命令后还会调用自动分片模块物化超长块；重放成功后门面更新命令栈、清空重做栈、标记未保存，组装新载荷返回；api 把载荷序列化为 JSON，server 写回响应；页面收到载荷后重建卡片、更新范围条与统计、把差异计数刷新。

这条链上共有五次模块边界穿越，每次穿越传递的都是普通字典与列表，没有共享可变状态，也没有回调。正因为如此，命令模块与对齐模块可以脱离 HTTP 单独测试，这也是第 11 章能对本项目做高密度单元测试的结构性原因。

第 7 章 关键算法设计

7.1 三套坐标与失配的根源

切片的正文不是原文的逐字子串，这是整个对齐问题的根源，也是一切算法设计的起点。自动切块链路依次做五件事：把超长单行展开成多个工作行、识别并掩蔽目次与前言区域、按文档家族解析出结构单元、把单元渲染成正文、按正文摘要去重后按长度拆成分片。渲染这一步会做清洗：图片语法替换为占位标记、空行删除、LaTeX 转义还原、加粗记号去掉、HTML 标签剥离、页尾省略号与续行标记清理。经过这一串处理，切片里的文字与源文件里的文字在空白、标点、标记符号上都可能不同，直接做子串查找的命中率为零，最初对一份七十四条切片的法规做全量查找，命中数为零。

由此产生了三套坐标。第一套是源行号，从 1 开始，指原始 Markdown 文件的行，界面显示与高亮都用它。第二套是工作行号，从 0 开始，指超长单行展开之后的行，对齐搜索在空间里进行。第三套是字符偏移，把原文按换行连接后从 0 开始计数，用于行内拆分与跨块合并时的位置表达。三套坐标之间的换算是确定的：源行与工作行之间靠一张映射表，只有被展开的行才有映射，其余行一一对应；字符偏移与源行之间靠前缀和，可以二分定位。

对齐算法的任务就是在这三套坐标之间建立切片到区间的映射，并对把握不足的情况如实降级。

7.2 规范化口径

对齐的前提是两侧用同一套规范化口径。源行与切片正文都经过同一个规范化函数：先剥离 HTML 标签，再把全角空格与不换行空格替换为普通空格，然后用正则把所有空白字符删掉，最后去首尾并转小写。表格行多一步：单元格先拼接，拼接前删除相邻重复的单元格，因为合并单元格在解析后会产生形如「1、1、阿片、阿片」的重复值，不去重就会与源行的单次出现对不上。源行侧还要先把竖线删掉，因为管道表的行以竖线分隔，切片正文里的表格行则不带竖线。

规范化的代价是丢失空白与大小写差异，这对法规文本是合适的：法规正文的空白多为排版产物，大小写差异只出现在英文缩写上。规范化之后，一条切片的正文行与源行之间通常是完全相等的关系，个别情况下是包含关系，即源行含有切片行（源行带列表符号或编号）。

7.3 三层对齐

对齐函数是纯函数，输入是切片列表、工作行数组与侧车字典，可选地传入映射表，输出是与切片等长的结果数组，每项含区间与状态。它按三层顺序尝试，前一层成功就不再进入后一层。

第一层处理条文、段落、标题与附录类块。它先把切片正文按换行拆开并逐行规范化，取第一个非空行作为锚点；锚点的数量可能不止一个，因为正文首行往往是面包屑，形如「区域 · 条号」，而面包屑在原文里可能出现在多处，因此还要把面包屑按分隔符切开取最后一段再规范化，用最具体的部分去搜。搜索从当前游标减二的位置开始向后线性扫描，允许完全相等与子串包含两种命中；若扫到文末仍未命中且游标大于零，则回卷到文首再扫一次，用于容忍文档中重复出现的表题与条文说明命中之后进入向后对齐：从命中位置的下一个工作行开始，依次为切片的后续每一行寻找匹配，搜索窗口限制在当前位置之后的二百行内，允许目标行等于某行或被某行包含。全部后续行都找到时，区间为「首个命中行到最后一个命中行加一」，状态为精确；若首行只是被包含而非相等，状态降为包含。

第二层处理表格块。它取侧车中的表格网格，把每一行拼成规范串，然后在一个不超过全表的窗口内寻找连续命中的行序列：从游标位置开始，逐行比对，命中则推进，未命中则跳过，连续命中达到行数要求即认为找到窗口。找到后区间覆盖该窗口，状态为表格行。表格块之所以不先走第一层，是因为表格正文的行在原文里可能是被压缩在一行中的 HTML，逐行搜索会全部落空。

第三层是表题锚定。当表格块前两层都失败时，它取侧车中的表号与表题，去掉空格后作为关键字，在游标减二之后的行里寻找含有该关键字的行，命中则区间为单行，状态为表题。这一层是为「表格被切成了多个块、但每块正文只包含表格的一部分」这种情况准备的：网格行匹配不上，表号却一定在表题行上。

三层都不成功时状态为未命中，区间为空。界面把未命中显示为灰虚线，不画行区间，卡片头部显示灰色的未对齐徽章。这一处理是刻意的：把未命中标到相近位置虽然能让界面「看起来完整」，但会让审核者基于错误的位置做判断，风险大于收益。

状态与含义的对应关系如下表。表中「近似」状态不由三层算法直接产生，而由编辑动作产生，表示该切片的区间是人工或合并推导出来的，不是对齐算法找到的。

状态	界面表现	产生条件
精确	绿色实线	正文首行与源行完全相等且后续行全部命中
包含	蓝色实线	正文首行被源行包含，或首行相等但后续行部分缺失
表格行	蓝色实线	表格网格的连续行走廊在原文中找到
表题	蓝色实线	以表号或表题关键字命中表题行
近似	黄色虚线	合并或扩展推导出的区间，未逐行验证
手工	灰色实线	人工编辑后由程序补定位的区间
未命中	灰色虚线	三层算法与兜底均失败，区间为空

7.4 相似度兜底

三层算法之后仍有少量切片未命中。观察发现这些切片的正文在原文中确实存在，但首行被改写或跨了行，例如条文被合并成一段、或原文在句子中间换行。为处理这一类，加入了一层基于相似度的兜底：在游标位置之后的六百行窗口内，逐行计算切片首行与候选行的相似度，跳过长度差超过「八与四分之一长度取大」的候选，相似度达到零点九三即认为命中，取相似度最高的候选作为起点再按第一层的方式向后对齐后续行，状态记为包含。

兜底的开销与命中率需要权衡。相似度计算是二次复杂度，六百行窗口对单条切片尚可接受，但若对每条切片都跑，整份文档的耗时会上涨数倍。实测表明加入兜底后，一份一百八十五条切片的文档由九十九条可定位提升到一百零四条，全量语料增加六条。收益不算大，但它处理的是最难的一类情况，因此保留，并通过「长度差预筛」与「仅对未命中项启用」两条限制控制成本。

7.5 三项改进的动机与效果

对齐精度在开发过程中做过一次集中改进，起因是全量语料的定位率停在百分之九十六点三。逐份分析未命中项后发现三个可归因的模式，对应三项改动。

第一项是整表单行复用。部分标准类文档把整张表压缩成一行 HTML，长度可达二十三万字符

原来的算法在命中后把游标推进一行，导致同一行上无法命中后续的表格块，因为这些块的行区间都是同一行。改动是：判断一行是否属于「超长行或含表格标签」，若是，则在首次命中后把游标停在该行而不是加一，允许后续块继续命中同一行。改动后一份一百八十五条切片的文档由一百六十条可定位提升到一百八十五条。

第二项是表题锚定。原来的算法只按正文行搜索，跨表的块（一块覆盖两张表的行）首行是表号而表号在原文里出现在表题行，与表格数据行不在同一行，于是整块落空。改动是增加第三层：用表号与表题在已知区块前后搜索表题行，以表题行的下一行到下一条表题之前为窗口重新整表匹配，全部行命中才采纳，区间从表题行开始。改动后跨表区间由九处降到五处，其中一处切片的区间由「第 724 至 731 行」修正为「第 727 至 731 行」，正好是表题向下到表尾。

第三项是相似度兜底，即上一节所述。

三项改动合计把全量语料的定位率从百分之九十六点三提升到百分之九十九点三二。最终统计为总切片 13 764 条，可定位 13 671 条；按状态分，包含 8 720 条、精确 716 条、表格行 4 206 条、表题 29 条、未命中 93 条。分文档看，消防法 74 条全部定位，国家标准 GB 50016 531 条全部定位（其中包含 466 条、表格行 64 条、精确 1 条），山东自由裁量基准 836 条全部定位（表格行 684 条、精确 146 条、包含 6 条），另两份地方标准分别为 104 条与 185 条，全部定位。

7.6 行级与字符级拆分

拆分命令有两个切法。行级切法接收切点行号，要求它落在一与行数减一之间，切完后上半块为切点之前的行，下半块为切点及之后的行。字符级切法接收切点字符位置，要求它落在一与长度减一之间，两半在同一行内切开。字符级切法主要用于单行块，例如一个长段落或一行表格，行级切法用于多行块。

拆分之后要解决两个问题：两半的行区间从哪来、下半块的条号取什么。行区间不能沿用原块的区间，否则两块会高亮同一段原文。做法是把原块区间作为窗口，在窗口内用规范化行重新定位：先定位上半块正文（取首行锚点加后续行顺序匹配），再从其结束位置向右定位下半块；若下半块在右侧找不到，则回退到整个窗口内再找一次。定位成功则写入区间，失败则保持空区间。条号按四级优先：命令显式给出、下半块正文首行能识别出条号、切点所在的源行能识别出条号、继承原块。第三级是为「两条合并成一块后再拆开」准备的，此时下半块正文首行可能只是条文内容，条号在源行的行首。

7.7 合并与扩展的原子性

合并有两种：两块合并与多块集体合并。两块合并校验相邻，允许命令里前后顺序颠倒，最终按文档顺序拼接正文；编号保留前块；条号与侧车在前块为空时取后块；区间取两块并集，只要两块都有区间就标为近似，否则置空并标为手工。集体合并校验编号列表无重复、索引连续，保留第一块的编号与类型，正文按文档顺序以换行连接，条号与侧车依次向后找第一个非空值。

扩展是合并的特例，用于「把选区覆盖的几块并入目标块」。它要求编号列表的第一项是目标块其余必须连续且不能跳块，正文可以由调用方给出（此时以选区原文为准），否则按顺序拼接。扩展完成后整体重排编号，因为扩展会改变块的数量与位置，重排能让编号继续表达位置。

三种动作的共同点是「一次命令、一次重放、一次撤销」。界面上的多选删除、集体合并、扩展都只发一条命令，撤销时也只弹一条，使用者按一次 Ctrl+Z 就能完整回退，不会出现「删了五块要按五次撤销」的情况。

7.8 自动分片的装箱

自动分片的目标是把超过阈值的块按语义边界拆成若干段，每段都带面包屑，且尾段不能太短。算法先摘出首行面包屑，再计算目标段数：总长除以阈值向上取整，且不少于二；每段目标长度为总长除以段数向上取整，且不低于最小长度。随后按行装箱：逐行累加，超过目标长度就换段；遇到单行本身超过目标长度时，先按标点硬切，切点从半段之后找最近的句号、分号或逗号，找不到就按长度切，切出的片各自独占一段，不与相邻行拼接。

装箱完成后做尾段重排：若尾段短于最小长度，则把它与上一段合并后重新分配，从上一段尾部搬行到尾段，直到尾段达到最小长度或上一段只剩一行。若上一段只剩一行仍不达标，则接受现状并

跳出循环。这里必须有一个跳出条件，否则会在「搬不动又反复尝试」之间死循环，代码中用「没有搬动任何行就恢复并跳出」实现。

最后给每段加上面包屑，生成编号（基础编号加 -p 序号）、分片序号与总数、重算派生字段，并把原块的行区间继承下来，使每个分片都能在前端定位到原文。极端情况下新编号可能与既有编号冲突（例如使用者先手工拆过一次），此时把冲突的分片编号顺延到最大序号之后，保证全局唯一。

表格块的分片规则不同：只在多行且超长时按行区间拆成多个独立表格块，每块写回「表 X 第 a-b 行」，单行超长的大表保持整块不拆。原因是表格的语义单位是行，按行拆不破坏列结构，而按字符拆会把一行切成两半，两侧都不再是合法表格。

7.9 混合块的按类型拆分

合并段落与表格会产生一种混合块：正文里既有段落文字，又有表格行，还可能夹着多张表的面包屑。当这样的块超长时，既不能按字符切（会把表格行切断），也不能整体按行拆（段落与表格会被打散到同一段里）。算法按类型边界拆：开头的标题类行归入首段头部；面包屑行是结构行，只用来切换「当前属于哪张表」，本身不进入正文；表格行按顺序归入当前表；段落行归入文本段。

判断「哪一行属于哪张表」靠面包屑而不是靠内容猜测。原因很实际：同一份文档里多张表的开头行可能完全相同（都是「风险点 · 编号：类型」这样的行），靠内容猜会把后面的表算到前面的表上。合并后的正文中，每张表的正文开头都会带一遍自己的面包屑，用面包屑切换是最可靠的信号；同一张表被拆成多个块且面包屑相同时，按出现顺序依次对应原始块。

拆出的表格块从原始块索引中找回行区间与侧车信息，因此条号仍能写成「表 X 第 a-b 行」；拆出的文本段若仍超长，再走一次文本分片。这样无论合并得多复杂，拆开后的结果都回到「段落是段落、表格是表格」的正常形态。

7.10 编号重排与差异统计

新增切片会触发编号重排，这带来一个副作用：差异统计若按编号比对，会把「整段顺延」误判为「删了十几条又加了十几条」。为此差异统计改用三段配对。第一段用内容签名（正文、条号、类型的组合）做最长公共子序列比对，完全相同的块视为未变动。第二段对剩余块按「条号加类型」配对，用于识别「正文改了但条号没动」的情况，配对成功的记为修改。第三段对仍未配对的块按正文相似度配对，相似度达到零点五即视为修改；为了避免大文档卡顿，当两侧剩余块数量乘积超过两千五百时跳过这一段。最后未被配对的当前块记为新增，未被配对的基线块记为删除。

三段配对的结果是三个数量与三组编号，界面显示为「加号、波浪号、减号」三个计数。使用者据此判断自己改了多少，而不必逐条比对。

7.11 草稿与基线的校验提交

草稿层的设计目标只有一条：保证提交时写进去的内容，是建立在这份基线之上的一次完整修订实现方式是把基线摘要写进草稿文件名：草稿命名为「文档名加基线文件摘要的前十二位加 .chunks.jsonl」。这样草稿与它的基线绑定，不需要额外的状态文件，也不会出现状态文件与草稿不同步的问题。

提交事务在校验之后才动正式文件，顺序为：确认草稿存在、文件名可解析、正式文件存在；重算正式文件摘要并与文件名中的摘要比对，不一致即中止并报「草稿已过期」；读取草稿并确认非空备份正式文件到备份目录，文件名带时间戳，轮转保留最近三份；把草稿裁剪成十五个契约字段，原子写入正式文件；删除草稿。任何一步失败都不会留下半成品，因为写入只有一次替换操作。

备份轮转的策略是「保留最近三份」，按文件名排序后删除多余项。这个数量是权衡的结果：太少无法回溯多轮修改，太多会占用空间且难以分辨。备份文件名带时间戳，人工恢复时按时间挑选即可。

7.12 复杂度与性能

对齐的主要开销是线性扫描。游标单调推进，每行的规范化只做一次并缓存，搜索在多数情况下只前进不后退，因此整份文档的复杂度接近「切片数乘以平均行数」。相似度兜底是唯一的二次开销但只在未命中项上启用，且带长度预筛。实测对齐一份五百三十一条切片的文档在一秒以内完成，全

部九十七份文档首次计算完成后进入缓存，再次打开在零点二秒以内。

编辑命令的开销与命令条数成正比：撤销是重放剩余命令，条数越多越慢，但单个文档的编辑通常在几十条以内，实测感知不到延迟。差异统计的最长公共子序列比对是平方级，两千五百对的上限把它挡在了可接受范围内。自动分片只在块超长时触发，装箱是线性的，唯一可能反复的是尾段重排代码用「搬不动就跳出」限制了迭代次数。

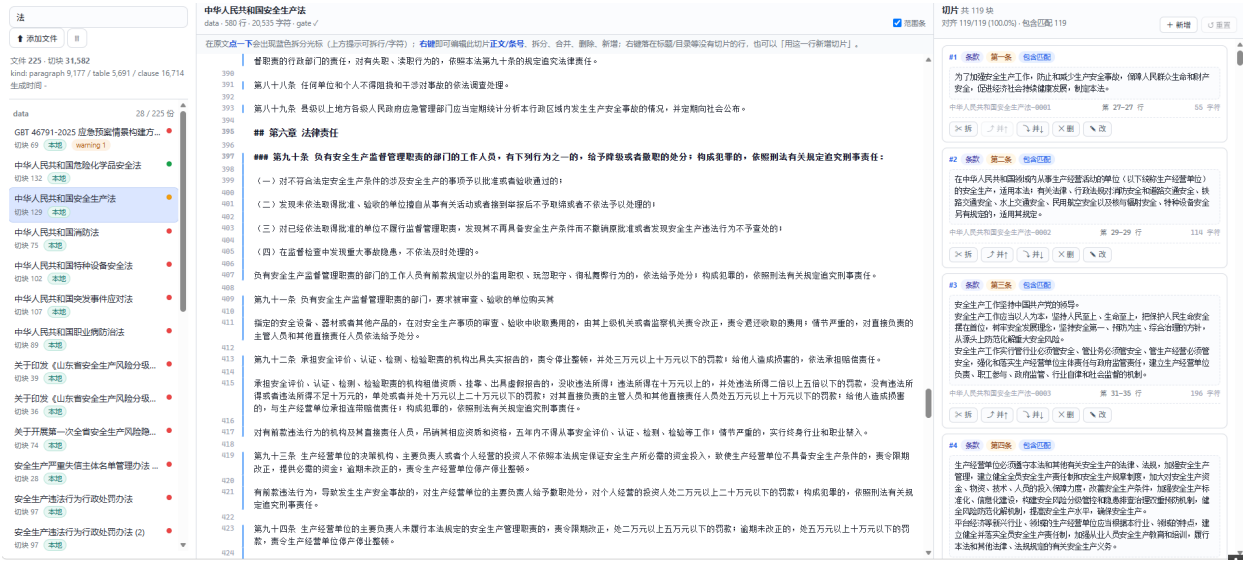
前端性能与算法无关，靠「一次构建加类名切换」保证：数据变化时整体重绘一次，交互阶段只改类名与位置，不重建节点。行数与块数越多，重绘越慢，这是本设计接受的代价，也是第 2.6 节所列边界的来源。表格的坐标定位按鼠标的实际位置换算到具体的表格行，滚动与选中都能对齐到行列这一点在超长表格上尤其重要，第 8 章会具体说明。

第 8 章　界面与交互设计

8.1 布局与尺寸

界面是单页应用，自上而下分三块：顶栏、三栏工作区、状态栏。顶栏与状态栏固定，不随内容滚动；三栏各自独立滚动，滚动条互不影响。三栏宽度用网格布局固定：左栏三百像素，中栏占据剩余空间，右栏在三百六十至四百八十像素之间伸缩，窗口变窄时按第 2.1 节所述的两级断点收窄并重排。宽度不做手工拖拽，理由是内容密度已经决定了合理区间——左栏只需容纳文件名与少量徽章，卡片栏超过四百八十像素后阅读效率反而下降，而中间原文需要尽可能多的水平空间，与其让用户维护一个宽度状态，不如按窗口宽度直接给出。这个取舍也省掉了一套拖拽状态与它的持久化。

三栏的边界用细线与浅底色区分，当前获得焦点的栏边框加深，避免使用者在密集文本中迷失位置。状态栏常驻显示三类信息：对齐统计（各状态的数量）、源文件校验状态、当前端口。状态栏的信息不随选中变化，只在文档重新加载后刷新，因此它适合用来判断「这份文档整体对齐得怎么样」而具体的对齐状态看卡片上的符号。



8.2 顶栏

顶栏从左到右分为四组。最左是产品标识，由方块图标、产品名与阶段标签组成，阶段标签随版本变化，用于区分不同阶段的产物。其右是当前文档的状态文本，格式为「已加载　法规名　· 切片数 · 对齐数/总数」，未选择文档时显示引导文案。

中间一组是编辑状态与撤销重做。草稿徽章有三种形态：有未保存修改时显示「● 有未保存修改」，草稿已保存未提交时显示「草稿已保存未提交」，草稿过期时显示「⚠ 草稿已过期」。未保存标记与草稿徽章并排，是因为两者含义不同：前者指内存中的修改尚未写入草稿文件，后者指草稿文件与当前会话的关系。差异计数显示为「加新增、波浪号修改、减删除」三个数字，鼠标悬停时提示「相对自动切块基线」。撤销与重做按钮在无对应历史时置灰，并带快捷键提示。

右侧一组是落盘与切块操作：保存草稿、提交（主按钮，蓝色）、放弃、导出、⚙ 切块、刷新

⚙ 设置。按钮的可用性跟着状态走：没有改动也没有草稿时保存草稿置灰；提交在有改动或有未过期草稿时可用；放弃在有改动或有草稿时可用；导出只在加载了文档后可用。切块按钮旁的监控标签在切块进行时显示进度，空闲时显示待切数量或上次结果，结果保留八秒后淡出，避免长期占用空间。

顶栏的高度固定，所有按钮为小尺寸，图标加文字，文字在窄屏下可省略但图标与提示保留。顶栏不做折叠菜单，原因是这些操作高频且需要一眼看到状态，藏进菜单会增加误操作概率。

8.3 左栏

左栏顶部是搜索框与添加按钮。搜索框占位文字为「搜索法规名 / 分组…」，输入即过滤，按文件名与分组名做不区分大小写的子串匹配，结果随输入实时收敛，清空输入即恢复全部文件；按Ctrl+K可以把光标直接跳到搜索框。添加按钮打开系统的文件选择框，允许一次选多个文件，扩展名限制为 Markdown 与纯文本。

搜索框下方是统计行，显示文件数与切片总数。再往下是文件树，按分组折叠。分组标题是子目录名，点击可收起或展开。每个文件占一行，左侧是文件名（过长时尾部省略），右侧是徽章区：切块数（形如「切块 185」）、来源标记（恒为「本地」，因为本版本只有本地数据源）、质检结果（通过显示 gate ✓，不通过显示 gate ✗）、警告数（有警告时显示「warning N」）、草稿标记（有未提交草稿时显示「草稿」，悬停说明「有未提交的草稿」）。徽章只反映状态，不做成按钮，避免在密集列表里误点。

文件行被选中后整行加深，右侧栏切换到该文档，状态栏刷新块数与对齐数。该文档有未提交的草稿时，左栏这一行带「草稿」徽章，顶栏的草稿徽章显示「草稿已保存未提交」，两张卡片上的内容就是草稿里的内容；草稿因基线变化而失效时，顶栏显示「⚠ 草稿已过期」，此时卡片栏显示的是正式基线的产物，而不是过期草稿里的编辑结果，避免让使用者以为改动还在。

左栏支持拖放。把文件拖到左栏时，整个左栏出现蓝色虚线边框与「松开导入到 data/source」的提示，松手后逐个上传，结果以提示条形式汇报创建、跳过与失败的数量。拖放与按钮走同一个上传接口，校验规则完全一致，不存在「拖进来能过、点按钮不能过」的差异。

8.4 中间原文窗格

中间窗格显示 Markdown 的源码行，而不是渲染后的富文本。这个选择在项目初期就确定，理由是源码行与文件行号一一对应，而渲染视图会因换行合并、列表重排导致行号无法回溯，定位与高亮都会失真。为了让源码可读，每种行按类型带上类名（标题、列表、表格、代码块、普通段落），由样式表控制字重、字号与颜色；行内不做 Markdown 解析，加粗记号、列表符号、引用符号都以原字符显示，这样任何一行的显示宽度都与字符数成固定比例，水平方向的位置计算才有依据。

每行左侧是行号槽，宽度固定，等宽字体，颜色浅灰。行号槽不参与编辑：点它不会出现拆分光标，也不选中切片，这样在滚动或拖选时不会因为蹭到行号而误触发。行号右侧二十像素是范围条，用于画切片覆盖的行范围；范围条可以整体关闭，开关状态记在浏览器本地存储中。点击正文区域才会出现拆分光标与提示气泡，右键则在光标位置弹出工作栏。

范围条是这一栏最有信息量的部分。它用竖直色条表示每个切片覆盖的行区间，颜色按切片序号轮换，相邻切片色相不同以便区分。多条切片在同一行重叠时（例如整表块与其内部的表格行块）使用多条泳道并排，最多四条，超过四条时合并为一根彩色条纹的聚合条，条纹颜色对应各切片。分片有双层表示：外层浅色条表示整个逻辑块覆盖的段落，内层深色条表示当前分片自身覆盖的行段，这样在多分片场景下能一眼看出「这一段被切成了四片，当前是第二片」。

原文行的底色表示当前命中的切片。悬停卡片时，对应行加浅底色；点击卡片锁定后，对应行加更明显的底色与左侧描边。未对齐的切片不画范围条也不画底色，行号槽对应位置显示灰色虚线；近似对齐的切片画黄色虚线。重叠区域使用斜纹，避免两种底色相互覆盖后产生第三种误导性的颜色。

超长单行是这一栏的特殊情况。标准类文档常把整张表压缩成一行 HTML，长度可达二十三万字符，若按普通文本铺开，浏览器既难以排版也难以阅读。界面的处理是识别出这一行里的表格，把它解析成真正的表格元素逐行渲染，行内因此出现的是几百行可见的表行，而不是一段二十三万字符的文本；同时该行在载荷里带截断标记，界面据此给它加一层折叠样式，并在这类行上放弃字符级定位，因为它们无法稳定映射回字符偏移。表格行仍然参与高亮与行选择，表格块的高亮落在整张表上。

表格在原文中直接渲染为真表格，而不是显示竖线与连字符。管道表按竖线切分单元格，HTML表按标签解析，两者都渲染为带表头、带斑马纹的表格。表格的每一行按所属切片着色：行底色淡，首列左侧有三像素色条，颜色由切片序号按黄金角旋转生成，因此相邻切片的颜色差异明显，一张九百多行的表可以出现三百多种色相而不重复。未切片的行显示灰色斜纹与首列虚线，明确表示「这一段没有对应的切片」。

8.5 右侧卡片

每张卡片对应一条切片，自上而下分四段。第一段是头部徽章行：最左是序号（形如 #12），随后依次是类型徽章（条款、表格、标题、段落等，按类型着色）、条号徽章、覆盖条号徽章（合并块覆盖两条以上时出现）、分片徽章（如「part 2/4 · 第 220–241 行」）、对齐状态徽章与手工徽章。徽章行把「这块是什么、属于哪一条、对齐到什么程度」三件事压在一行里，卡片纵向因此更紧凑，八百多条切片时也能快速扫读。第二段是正文区：优先按表格网格渲染，其次是纯文本或分段渲染，规则见下文。第三段是脚注行：等宽的切片编号、覆盖的原文行区间（未对齐时写「未对齐」）、字符数。第四段是操作行：拆、并↑、并↓、删、改五个按钮，首块禁用「并↑」，末块禁用「并↓」。

正文区的渲染方式由数据决定。切片在侧车里带表格网格时按网格渲染，网格之后附一个可折叠的「原始正文」，说明送进检索的就是折叠里的这段文字；带网格的块若被人工改过正文，则不再使用旧网格，避免显示与内容对不上。切片没有网格、但覆盖的原文位置上有表格时，先显示正文，再把原文里的表格整块克隆下来渲染，标题为「本切片覆盖的原文表格 · 第 N 行」，克隆时清除高亮类名与行内样式，这段渲染只读，不写入数据。既无网格也无覆盖表格时，正文按段落与表格分段：能识别为管道表或 HTML 表的片段渲染成真表格，其余文本用等宽字体呈现。

头部徽章里有一类是从原文行倒推出来的，即覆盖条号徽章。合并后的大块横跨多条时，卡片会遍历它覆盖的原文行，收集其中出现的条号与层级号，若收集到两个以上就显示「覆盖 第一条 – 第三条（3 条）」。这个徽章是纯展示，不回写契约，原因是条号字段要参与检索与排序，塞入区间字符串会污染检索结果，而覆盖关系用行区间已经可以精确表达，没有必要在契约里重复一份。同理，分片徽章里的行段、对齐徽章里的状态文字，也都由视图层按当前数据计算，不进入导出的字段。

表格块另有一套展示。若侧车中有表格网格，卡片按网格渲染真表格：表头行按记录的层数加底色，合并单元格按解析结果抄平为多列，注释行（从记录的行号起）用浅黄底色，列宽取记录里的宽度数组，网格有多少行就渲染多少行，不做行数截断，因为一屏之内看到完整表格比省几个节点更重要。网格只负责「看起来像原文」，检索与导出的仍然是正文，所以网格下方始终提供那段可折叠的原始正文。若一条切片既包含正文又覆盖表格（合并段落与表格后可能出现），卡片先显示正文，下方再按原文克隆渲染它覆盖的表格，这段渲染不写入数据，刷新或重新加载后仍能重建。

卡片列表一次性构建，滚动时元素始终留在文档里，因此选中状态不会因为滚动而丢失。选中由数据里的编号决定，而不是由节点是否「在视口内」决定：每次重绘后按编号找到卡片，补上选中类名。卡片被悬停时边框变为浅蓝，同时原文对应行高亮；点击卡片任意位置即选中该切片并把原文滚到它的首行，Ctrl 或 ⌘ 加点击则切换多选，状态栏提示已选数量与可用操作。

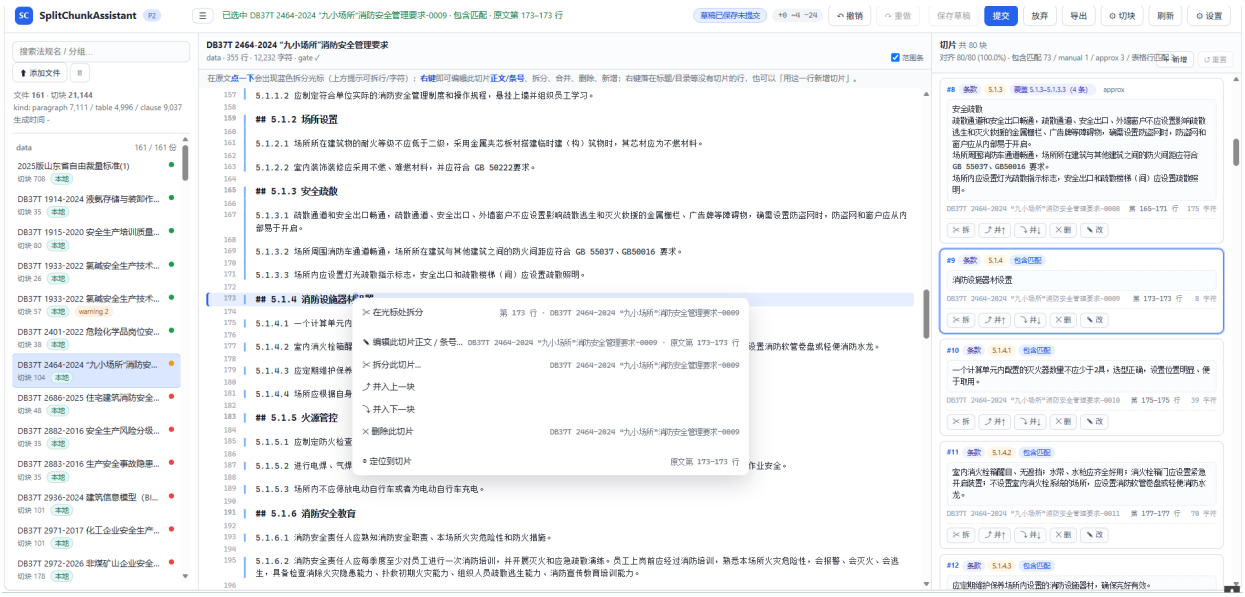
8.6 弹层与浮层

界面共有五个对话框：设置、拆分、编辑、新增、选择。设置对话框列出八个配置项，每项带字段名、说明与输入控件，底部显示校验错误，保存后立即生效（端口除外）。拆分对话框列出该块的所有行，每行前有单选按钮，用于选择「在哪一行之前切开」；单行块改为显示字符位置输入框；底部有一个可选的「下半块条号」输入框，提示文字说明留空时会自动识别。编辑对话框提供正文文本域、条号输入框与类型下拉。新增对话框与编辑类似，多一个来源说明（选区、表格行或手工输入）条号留空时提示将填「（新增）」。选择对话框用于需要三选一的场合，例如删除分片时询问「只删这一片、删除整组、取消」。

浮层有三类。第一类是右键菜单，在原文、卡片或表格行上右键时出现，菜单项按上下文变化。第二类是拆分光标气泡，在原文点击后出现，显示「第 N 行」「行边界」或「光标处」以及目标切片编号，若该位置不可拆则显示原因而不是按钮。第三类是提示条，出现在页面底部，用于报告操作结果与错误，以不同颜色区分成功与失败，显示二点六秒后自动消失；状态栏则保留最后一次操作的

结果，不会自动清空，方便回看。

所有对话框使用浏览器原生的 dialog 元素，因此自带焦点陷阱与 Esc 关闭行为，减少了自己实现焦点管理的风险。对话框打开时背景加半透明遮罩，点击遮罩不关闭（避免误触丢失编辑内容），必须点取消或按 Esc。



8.7 表格交互

表格行的选中不使用浏览器的文字选区，而是自维护一套行带选区。原因是文字选区在行高很大（实测有二百八十五到四百四十像素的行）或单元格内含块级元素时，边界会落在单元格内部，无法稳定映射到「第几行」。自维护选区的做法是：按下鼠标时用坐标定位到具体的表格行，记录起始行移动时按坐标扩展，跨行时整行选中；松开时若起始行与结束行相同则视为单击。这样无论行多高、单元格多复杂，选区始终以整行为单位。



定位的具体做法是先取鼠标位置命中的单元格，再沿父链找到最近的行元素，行元素上带有切片编号与行序号两个数据属性。这条路径兼容合并单元格：合并单元格跨越三行时，命中的是同一个单元格，但沿父链找到的行是发起合并的那一行，行序号仍正确，因此不会出现「点第三行选中第一行」的错位。实测对第 120 行与第 300 行的点击分别定位到编号 0239 与 0373 的切片，与数据一致。

选中后按以下规则反馈：选中的行加琥珀色底色与左侧竖条；状态栏显示「表格内第 a–b 行 · 已选 N 行」；右键菜单按选区内容变化——选区全是已切片行时提供合并与删除，含未切片行时提供「用选中的表格行新增切片」，两者混合时提供新增为主、其余置灰。按住 Shift 点击可以扩展选区

按 Esc 或点击表格外区域清除选区。拖动到表格上下边缘时自动滚动，滚动速度随距离边缘的远近变化。

未切片行是重点保护对象。它们在视觉上用灰色斜纹与首列虚线区分，单击或拖选时变为琥珀色右键第一项是「用这一行新增切片」。新增时的插入位置按表格行顺序确定：新切片插在它上方最近一个已切片行所属切片之后、下方最近一个已切片行所属切片之前；若上方没有已切行，则插到该表格第一个切片之前。预填条号时会参考上下相邻的表格切片，用「表号前缀加行区间」与「表内行号偏移」两条线索交叉验证，只有结果严格落在两个邻居之间才采纳，并在弹层里提示「条号已预填，请确认」，不静默写入。

表格滚动位置的保持是另一个细节。编辑后页面会重建卡片与表格，若不做处理，滚动位置会跳回顶部，使用者在长表格中修改一行后很难找回原来的位置。界面的做法是关闭浏览器的滚动锚定，记录操作前的锚点（表格块起始行或卡片编号），重建后按锚点恢复。实测在三千六百像素的滚动位置上删除第 700 行的切片，恢复误差为零像素。

8.8 快捷键

快捷键覆盖最常用的十个动作，全部在按钮的悬停提示中标注。快捷键在输入框获得焦点时失效避免打字时误触发。

快捷键	动作	说明
Ctrl+Z	撤销	撤销最后一条编辑命令
Ctrl+Y 或 Ctrl+Shift+Z	重做	重做刚撤销的命令
Ctrl+S	保存草稿	立即写草稿，不等防抖
Ctrl+Shift+N	新增切片	在选中块之后新增
Ctrl+Enter	拆分当前块	打开拆分弹层
Delete	删除选中块	选中状态下直接删除
Ctrl+K	聚焦搜索	光标跳到左栏搜索框
Esc	取消	关闭弹层、取消锁定、清除表格选区
Ctrl+点击卡片	多选切片	切换该卡片的选中状态，选中后可一次删除
Shift+点击表格行	扩展行选区	以起点行与当前行为界扩展表格行选区

8.9 视觉与状态色

视觉规范以可读性优先。字体在 Windows 上使用系统界面字体，等宽部分使用 Consolas；正文十四像素，行高一点六五倍；主色为深蓝灰，强调色为蓝，危险操作为红。切片的八种底色为低饱和轮换色，保证在白底上长时间阅读不刺眼，同时相邻切片可区分。对齐状态色固定：精确为绿，表格行与包含为蓝，近似为黄，未命中为灰红。状态色不参与切片的身份识别，身份识别只看编号与范围条颜色。

配色只有一套浅色主题，不提供主题切换。这样处理的原因是切片的八种底色、四种对齐状态色都按浅色底校准过对比度，换成深底需要重新调色并重新验证可读性，而界面的主要使用场景是白天办公环境，收益不足以支撑一套并行的配色。打印样式未做专门适配，因为界面不是交付物，交付物是导出的 JSONL。

8.10 可访问性与容错

主要区域都有语义标签，三栏分别为文件列表、原文、切片，按钮有文字或提示，图标不是唯一的信息载体。对话框使用原生元素，键盘可完整操作。表格行的选中提供状态栏文字反馈，不只依赖颜色。

容错方面，界面在每个可能失败的位置给出具体原因而不是通用错误：上传失败会说明是编码问题、大小超限还是文件名非法；拆分失败会说明切点越界与可取范围；提交失败会说明是草稿过期还是正式文件缺失。任何失败都不会改变当前编辑内容，使用者可以修正后重试，不必从头再来。

8.11 界面层的性能策略

界面层的性能策略可以归纳为四条。第一，一次构建、多次复用：整份文档的行与全部卡片在数据变化时构建一次，交互阶段不再触碰节点结构，只改类名与滚动位置。第二，不做整树重建：联动选中、高亮都通过切换类名实现，类名切换不触发布局重排，也不会让文本框失焦。第三，合并高频

事件：滚动与鼠标移动触发的滚动请求用动画帧合并，一次帧内只滚一次，避免长列表上的滚动风暴第四，把重活放到后端：对齐、分片、差异统计、编号重排都在后端完成，界面只做展示与命令拼装因此前端脚本始终保持在三千八百行左右，没有出现难以维护的渲染逻辑。

这四条策略带来的可见效果是：九百行表格的行选中与着色即时响应，编辑后重建页面保持原滚动位置，快速拖动滚动条不出现空白。相对的代价集中在首次渲染：构建成本与行数、块数成正比，行数上万时需要可感知的等待时间，且内存占用随元素数量增长。这一代价被明确接受，因为本软件面向的是单份几十到一千行的法规文本，而不是动辄数万行的大型文档；若将来要支持后者，把行与卡片改为窗口化渲染是首选方案，届时只需改动两处渲染入口，不影响其余分层。

第 9 章 操作流程

9.1 流程总览

一次完整的使用过程可以概括为八个步骤：启动程序、准备语料、选择文档、核对对齐、执行编辑、保存草稿、提交落盘、导出或重切。这八个步骤并非每次都要全走一遍，日常使用中最常见的是「启动、选文档、改几处、提交、关闭」。本章按步骤顺序说明每一步的操作方式、界面反馈与注意事项，并在最后给出一条从零开始的完整示例。

需要先建立的一个认识是：编辑始终发生在内存与会话中，草稿只是自动的中间落盘，正式产物只在提交那一刻被替换。因此任何操作都可以放心尝试，最坏的结果是按一次放弃回到基线，不会破坏原始数据。理解这一点之后，使用者可以大胆地拆分与合并，而不必担心「改坏了怎么办」。

9.2 启动与进入

双击根目录下的 run_studio.cmd，或在命令行执行模块启动命令。控制台会依次打印启动信息：第一行是版本与访问地址，随后是数据目录、切块引擎（不可用时改为一行警告），最后是文件数与切片条数，随后浏览器自动打开工作台。若浏览器没有自动打开，把控制台打印的地址粘贴到地址栏即可。

如果控制台提示该端口已有本软件在运行，说明之前启动的实例还在，程序已经直接打开那个实例的页面，此时不需要再做任何操作。如果提示端口被占用，按提示换一个端口重新启动，例如在原命令后加「--port 8801」。启动后不要关闭控制台窗口，关闭窗口等于停止服务，页面会失去连接。

9.3 准备语料

语料有两种准备方式。第一种是把 Markdown 文件直接放进数据目录的 source 子目录，可以放在一级子目录中，子目录名会成为左栏的分组名。第二种是在界面里导入：点左栏的「⬆ 添加文件」按钮选择文件，或把文件直接拖到左栏区域，拖入时会出现蓝色虚线框与「松开导入到 data/source」的提示。

导入有几条硬性规则需要知道。只接受 Markdown 与纯文本，纯文本会被另存为 Markdown 扩展名。文件必须是 UTF-8 编码，若原文件是 GBK 或 GB18030，会提示「不是 UTF-8 编码（请先转码）」，需要先用编辑器转换编码再导入。同名文件不会被覆盖，只会被跳过并在结果里报「已跳过 N 个」。单个文件不超过十六兆，一批不超过六十四兆。文件名不能含路径分隔符与控制字符。

导入成功后，若开启了自动切块，程序会在数秒内扫描到新文件并开始切块。顶栏右侧的状态标签显示「切块中 1/1 · 文件名」，完成后变为「已切块 N 份」。切块失败的文件在左栏显示警告标记切片数为零，最常见的原因是编码不对或文件内容为空，控制台会打印引擎的错误输出。

9.4 选择文档与初读

点左栏任意文件，中间显示原文，右侧显示切片，状态栏刷新对齐统计。第一次打开一份文档时建议先看三个地方。第一看状态栏的对齐统计，如果未命中数量明显偏多（比如超过总数的百分之五），说明这份文档的表格或版式特殊，后续阅读时要留意灰色虚线的切片。第二看左栏该文件的质检徽章，通过显示对勾，不通过显示警告，点开提示可以看到失败项。第三看切片总数与原文行数是否匹配预期，若切片数为零而原文有内容，说明切块没有成功。

阅读阶段的主要动作是核对边界。默认的高亮是「相邻切片不同色」，所以一眼就能看出哪些行属于哪一条切片，边界是否落在条文的条、款、项之间。若文档较长，可以先用左栏搜索定位到目标

法规，再在原文里滚动到相应条号；本版本不提供目录跳转，跳转依赖切片与原文的双向定位，这是刻意的取舍——目录需要额外的解析与锚点维护，而切片高亮已经把「哪一段属于哪一条」直接画在了原文上。

9.5 阅读联动

联动有三种触发方式。悬停是最轻的一种：把鼠标放在卡片上，原文对应行加浅底色；把鼠标放在原文行上，右侧对应卡片加边框。悬停不改变选中状态，移开即恢复，适合快速确认「这条切片在哪」。

点击是锁定：点卡片的任意位置，该切片被选中，原文滚动到它的首行并加描边；点原文某行，右栏滚动到覆盖该行的第一张卡片。锁定的状态下再点一次取消，按 Esc 也可以取消。锁定的意义在于「一边看原文一边改卡片」时不会因为鼠标移动而丢失位置。

拖选是反查：在原文中拖过一段文字，右侧所有覆盖这段文字的卡片会被标记，界面上方显示「原文第 a–b 行 → 对应 N 个切片」。如果选区里包含没有切片覆盖的行，提示会如实说明「没有对应的切片」，不会假装全部覆盖。这个动作在核对「这一段到底被切成了几条」时最有用。

范围条提供了第三种视角：它把整份文档的切片分布画成一条竖条，能一眼看出哪里切片密集、哪里有大段空白。把鼠标放在色条上会显示该切片的编号与行区间，点击即选中。对于整表块，范围条按表格的整块高度竖向铺满，看起来是一根很长的色条，这与它在原文中只占一行的事实并不矛盾色条的长度表示它覆盖的行数，而这一行在原文里已经展开成几百行可见的表行，两者是同一件事的两种画法。

9.6 拆分

拆分有三个入口。第一，卡片上的「✂ 拆」按钮，打开弹层后选择切点行。第二，在原文某行点击出现蓝色光标后右键，选择「在光标处拆分」。第三，选中卡片后按 Ctrl+Enter。三个入口最终都走同一个命令，行为一致。

弹层里列出该块覆盖的所有行，每行前有单选按钮，文字为该行的前若干字符。选择「在第 N 行之前切开」后，上半块保留原编号与原条号，下半块获得新编号。若两半的条号不同，可以在弹层的「下半块条号」输入框里填写；留空时程序会自动识别切点处的条号，识别顺序是「下半块正文首行、切点所在源行、继承原块」。对于单行块，弹层改为显示字符位置输入框，输入一与长度减一之间的整数即可在行内切开。

拆分完成后，两半都会重新获得原文行区间，右侧出现两张卡片，原文对应位置的高亮分成两段若某一段找不到行区间（例如切点落在被清洗掉的空白处），该段的区间为空，卡片上显示未命中状态，此时可以用「扩展」或手动重划区间修正。拆分可以连续进行，一条长条文可以拆成多条，每次拆分都可单独撤销。

有一点需要提醒：拆分产生的新编号取当前最大序号加一，因此新块在列表中排在末尾，而不是紧跟在原块之后。这是刻意的取舍——编号代表身份，不随位置变化，列表位置由行区间决定。界面按行区间排序显示卡片，所以视觉上新块仍在原块附近。

9.7 合并、集体合并、扩展与并入

两块合并用于「切成两条但其实是一条」的情况。卡片上有「⤴ 并↑」与「⤵ 并↓」两个按钮分别与上一块、下一块合并。合并后保留前一块的编号与条号，正文按顺序拼接并以换行分隔，行区间取两块并集并标为近似。

集体合并用于「一次并很多块」。在原文中拖过要合并的几块，右键选择「合并所选 N 个切片」，要求这些切片在列表中连续；合并后保留第一块的编号与类型，正文按文档顺序连接，若合并结果覆盖了多个条号，卡片上会出现「覆盖 第一条 – 第三条（3 条）」的徽章。

扩展用于「选区比原块大，想把多出来的部分并进原块」。操作方式是从某块的第一行行首开始拖选，一直拖过该块末尾继续向下选，松手后右键，第一项会变成「扩展切片」。与合并的区别是：扩展不新建块，而是把目标块的行区间变长、正文变成选区原文，块的总数减少（被并掉的块消失）文字与表格行都支持扩展，表格行扩展时以表格行为单位，不涉及段落。

并入用于「原文有一处没有切片，但内容属于下面那条」。从没有切片的位置开始拖选，一直拖

进下面某个切片，右键第一项是「把未切片选区并入 #N 切片」。执行后块的总数不变，目标块的行区间向前延伸，覆盖到选区起点。这个动作与「新增」的区别在于不产生新块，适合处理「上一段开头被漏掉」的情况。

四种动作都是一条命令、一次撤销。合并与扩展之后，差异统计会显示修改数增加，而不是「删了又加」，因为配对是按内容签名做的。

9.8 删除

单块删除有三种方式：卡片上的「✕ 删」按钮、选中后按 Delete、多选后一次删除。普通整块删除不弹确认，因为可以撤销；这一点与很多软件相反，但配合「删除后不重建页面、只重排卡片」的实现，误删的恢复成本极低。

删除分片时要多问一句。若被删的块属于一个多分片组，弹层会给出三个选项：只删这一片、删除整组、取消。选择只删这一片后，同组剩余的片会重新编号，若只剩一片则去掉分片后缀恢复为普通块。若选区中包含同组的部分分片但不是全部，弹层会改为提示「只删选中的、连同整组一起删、取消」，避免出现「半组缺失」这种语义不完整的中间状态。

多选的方式是按住 Ctrl 点击卡片的编号，或在原文中拖选后右键选择「删除所选 N 个切片」。多选删除只产生一条命令，按一次撤销即可全部恢复。实测一次删除两条切片再从备份外恢复的操作为：删除后块数由八百三十六变为八百三十四，按一次撤销恢复为八百三十六。

9.9 修改与新增

修改针对单块的正文、条号与类型。点卡片上的「✎ 改」打开弹层，正文文本域可编辑，条号输入框接受任意文本但会做宽松校验（格式异常只提示不阻断），类型下拉限定在七个枚举值内。保存后若正文发生变化，长度与摘要会重算，该块被标记为手工并与原文区间脱钩，卡片上的对齐状态变为灰色。只改条号不影响正文与派生字段，也不影响行区间。

新增有三种来源。第一，在原文中拖选一段文字后右键选择「用选区新增切片」，选区内容直接成为正文，行区间自动记录。第二，在没有切片的行上右键（例如标题行、目录行），选择「用这一行新增切片」。第三，在表格中拖选未切片的行，右键选择「用选中的表格行新增切片」，正文按「列名：值」的格式拼接，类型默认为表格。

新增弹层会提示插入位置与编号策略。若选区在原文中没有切片覆盖，插入位置由选区所在位置决定；若选中了某块，则插入到该块之后。条号留空时按前后邻居倒推：先前一块的条号，再后一块的条号，两者都识别不到时填「（新增）」。从表格行新增时，条号会按上下相邻的表格切片预填，并在弹层里提示「条号已预填，请确认」，使用者可以修改后再保存。

新增之后编号会按位置重排，插入点之后的切片编号顺延，因此差异统计会出现较多「修改」，这是正常的，界面不会把它当作错误。

9.10 草稿

编辑发生两秒后，界面自动把当前块列表写入草稿文件，顶栏徽章由「未保存」变为「草稿已保存」。按 Ctrl+S 可以立即保存，不等防抖。切换到另一个文件时，当前文件若还有未保存的修改，会先补存一次再切换。

草稿徽章有三种状态，含义不同。「未保存」表示内存中的修改还没写进草稿文件，通常只持续两秒。「草稿已保存」表示草稿文件已是最新，但还没有提交到正式产物。「草稿已过期」表示草稿所基于的基线已经变了，常见于源文件被外部修改并重新切块之后，此时草稿不能提交，只能放弃后重新编辑。

草稿文件按文档分开存放，文件名中内嵌基线摘要，因此同一文档在不同基线下可以有不同草稿但实际只会保留最新一份。关闭浏览器、关闭程序、重启电脑之后再次打开，草稿仍在，界面会直接加载草稿内容并在左栏显示「草稿」标记。这一点是草稿机制的主要价值：编辑不必一次做完。

9.11 提交、放弃与导出

提交是把草稿提升为正式产物的动作。点顶栏的「提交」后，程序先补存一次草稿，再重新读取正式切片文件并计算摘要，与草稿文件名中的摘要比对；一致则把正式文件备份到备份目录（保留最

近三份），把草稿裁剪成十五个契约字段后原子替换正式文件，删除草稿，重新加载文档。界面上会出现一条提示，说明提交了多少块、备份文件叫什么。

提交失败最常见的原因是「草稿已过期」，处理方式是放弃草稿后重新编辑。其次是正式切片文件不存在（被人为删除），此时同样需要放弃草稿并重新切块。提交过程中不会出现「只写了一半」的状态，因为写入是原子的。

放弃是把草稿删掉、回到基线的动作，会二次确认。放弃之后当前会话中的编辑也一并清除，界面显示自动切块的结果。若只是想让某几处修改失效，不要用放弃，用撤销更精确。

导出是下载当前块列表为 JSONL 文件，包含尚未提交的修改。导出的文件名带文档名与时间戳内容为十五个契约字段，每行一个对象。导出不改变任何状态，适合在提交前把修改发给他人复核，或在不希望改动正式产物时另存一份。

9.12 重新切块与质检

手动切块通过顶栏的「⚙ 切块」按钮触发，它会扫描源目录中还没有切片的文件并排队切块；也可以只针对某个文件触发，此时若无变化会提示「选中的文件没有变化，不需要重切」。切块过程在后台线程中进行，界面上的状态标签显示进度，完成后的结果保留八秒。

若源文件被修改（摘要与切块时记录的不一致），左栏该文件会显示「过期」标记。默认不会自动重切，因为重切会重排编号，使基于旧编号的草稿与人工修改失效。需要重切时的正确顺序是：先提交或放弃草稿，再关闭自动切块开关，删除该文档的产物目录，最后手动触发切块。这个顺序在使用说明中被反复强调，原因是跳过其中任何一步都可能造成修改丢失。

质检结果来自切块时引擎写出的报告，界面只做展示，不提供「重新质检」按钮。若需要复查，重新切块即可得到新的报告。质检不通过的文档会在左栏显示警告，点开可以看到失败项的名称与数值，便于判断是语料问题还是切块参数问题。

9.13 一条完整示例

以一份地方规范性文件为例，它有一千余行原文、八百三十六条切片，其中大量表格块被切得偏碎。使用者先启动程序，导入文件，等状态标签显示「已切块 1 份」。打开该文件，状态栏显示对齐统计，未命中为零。

阅读时发现同一张表的表头与表体被切成两条，于是从表头行行首拖选到表体末尾，右键选择「合并所选 2 个切片」。卡片变为一条，编号取前一块，覆盖条号徽章显示「覆盖 第二部分–第二部分（2 条）」。随后发现有一行表格内容没有被任何切片覆盖，在表格中拖选该行，右键选择「用选中的表格行新增切片」，弹层里条号已按上下邻居预填为「表 1 第 215-215 行」，确认后块数由八百三十六变为八百三十七。又发现一处段落的开头被漏掉，从漏掉的位置拖选进下面那条切片，右键选择「并入 #N 切片」，块数不变，该块行区间前移。

两秒后顶栏显示「草稿已保存」，左栏该文件出现「草稿」标记。使用者关闭浏览器去处理别的事，第二天重新打开，草稿自动加载，界面提示这是草稿。核对无误后点「提交」，提示提交成功、备份文件为「文档名.20261105-111654.jsonl」。刷新页面，卡片仍是修改后的结果，状态栏的对齐统计已按新块重算。最后点「导出」留一份 JSONL 存档，流程结束。

第 10 章 异常处理与边界条件

10.1 异常处理原则

本软件的异常处理遵循四条原则。第一，不静默失败：任何失败都要在界面上给出具体原因，而不是只留一行控制台日志。第二，不写半成品：会改动磁盘的操作都采用「临时文件加原子替换」，要么完整写入，要么保持原样。第三，可回退：编辑类失败不改变已有状态，编辑成功后可用一次撤销还原。第四，错误可定位：错误信息尽可能带上文档名、块编号、字段名与取值范围，使用者不必猜。

这四条原则落到实现上，形成了一套分层处理：底层函数只抛异常，不做界面提示；接口层把异常翻译成状态码与文案；界面层把文案显示在提示条或弹层里，并且不改动当前编辑内容。这样的分工使得同一条错误在不同入口下表现一致，也使得单元测试可以直接断言异常类型与信息，而不需要

模拟浏览器。

10.2 对齐失败

对齐失败（未命中）不是程序错误，而是数据本身的边界情况，因此不抛异常，只标记状态。未命中的切片在界面上显示灰色虚线与叉号，不画范围条，卡片头部显示灰色的未对齐徽章，脚注行写「未对齐」，选中它时原文窗格不会滚动到任何位置。使用者可以采取三种处理：一是核对正文是否被切块引擎改写过度，若正文已经不可读，删除该块；二是用「扩展」或「新增」把它的内容并到相邻块里；三是重新切块，看新的产物是否能定位。三种处理都不修改原文，只在覆盖层生效。

未命中率是判断文档质量的指标。单份文档的未命中率超过一成时，界面在左栏用黄色圆点提示建议在提交前逐条核对。全量语料的未命中率约百分之零点七，主要集中在两类文档：把多张表压缩成超长单行的标准类文本，以及带有复杂合并单元格的表格文档。这两类问题在开发中已经做过针对性改进（整表单行复用、表题锚定、相似度兜底），剩余部分需要切块引擎提供行号来源才能根治，属于后续阶段的工作。

10.3 草稿过期与基线漂移

草稿过期是最常见的使用性错误。触发条件是：草稿文件名中的基线摘要与当前正式切片文件的摘要不一致。造成不一致的原因有四种：源文件被外部修改并重新切块；手动删除了产物目录并重切另一台机器或另一个会话提交了该文档；备份恢复操作替换了正式文件。

发现过期的方式有两种。界面加载时会比对并显示「⚠ 草稿已过期」，此时草稿内容不会被加载，卡片显示的是基线内容。提交时服务端会再次比对，不一致则拒绝提交并返回「草稿已过期（基线变了），请放弃草稿后重新编辑」。

处理方式只有一条正确路径：放弃草稿，重新编辑。没有「强制提交」选项，这是刻意的：强制提交会把基于旧基线的编号与行区间写进新基线，造成编号冲突与高亮错位，后果比丢失一次编辑严重。若使用者确实需要保留旧草稿的内容，可以先导出 JSONL 存档，再放弃，然后把需要的内容手工重建。

预防措施有三条。编辑前先提交或放弃上一次的草稿；不要在编辑过程中修改源文件；开启自动切块时，改源文件前先关闭该开关。使用说明中把这三点写成了一条显式警告。

10.4 校验类错误

编辑命令在重放时会做严格校验，不通过即整条命令失败，状态保持不变。常见的校验错误如下表。表中「错误文案」为界面实际显示的文本片段，使用者可以据此在源码中定位校验点。

触发条件	错误文案片段	处理方式
切点行越界	切点行号超出范围（可取 1 至 N）	改选合法行或改用字符切分
切点字符越界	字符位置超出范围（可取 1 至 N）	改填合法位置
合并非相邻块	只能合并相邻的两个切片	检查中间是否还有块
集体合并不连续	所选切片不连续	改为连续选区或分批合并
删除不存在的块	找不到切片：编号	刷新页面后重试
新增正文为空	新增切片正文不能为空	输入内容或改用选区
类型非法	未知的切片类型	从下拉中重新选择
分片序号越界	分片序号不能大于总数	修正序号或先规范化分片
编号重复	切片编号重复	高级模式下修正编号
表格块拆成分片	表格块不支持分片	改为按行拆分

这些校验的价值在于把错误挡在状态变更之前。由于校验与执行复用同一段重放代码，不存在「校验通过但执行到一半失败」的情况；一条命令要么整体生效，要么整体不生效。界面上，失败的命令不会产生任何可见变化，只在提示条里说明原因，使用者修正后重试即可。

10.5 数据与编码类边界

编码是最常见的导入失败原因。程序只接受 UTF-8，检测方式是按 UTF-8 严格解码，失败即报「不是 UTF-8 编码（请先转码）」。之所以不做自动转码，是因为自动猜测编码在中文文本上错误率不低，猜错会产生乱码且难以察觉，宁可让使用者显式转换。带 BOM 的 UTF-8 可以正常读取，读取时会去掉 BOM。

空文件与纯空白文件在导入时被拒绝，提示内容为空。已经存在于数据目录的空文件会被列在左栏，切片数为零，质检不通过。源文件缺失而产物存在时，文档仍会列出，但打开时返回 404 并提示源文件缺失，卡片栏为空；这种情况多发生在只拷贝了 chunks 目录或误删源文件之后，处理方式是把源文件放回原位或删除该产物目录。

超长行是另一类边界。单行长度超过两千字符即被视为超长行，对齐时按「整表单行」处理：整张表在源文件里只占一行，所有从它切出的切片共用这一个行号，界面因此按表格行而不是按文本行着色。单行超过二十三万字符时，这一行会被解析成几百个表行渲染，首屏要为它多花一次排版时间实测可以接受，但滚动与重绘明显比普通段落慢。这类行上不提供字符级定位，只能整行选中，因为它们无法稳定换算回字符偏移；这是当前契约的已知限制，精确到行需要上游在产物里给出行号来源。

超大文件的上传上限是单文件十六兆、整批六十四兆。超过上限的请求在传输层就被拒绝，返回 413 与「请求体过大」。这个上限对法规文本足够宽松，一份百万字的法规约两兆，正常使用不会触达。

10.6 服务与进程类异常

端口占用与实例重复是启动阶段的两类异常，第 3 章已说明处理方式。这里补充引擎相关的三类

第一类是引擎缺失。表现为启动时控制台打印「[!] 切块引擎不可用」并给出原因，界面仍可打开并浏览既有切片，但无法自动切块。原因是 vendor 目录被删或打包时漏收，处理方式是把引擎目录放回 vendor\splitchunk，或在设置里填写正确的引擎目录。

第二类是引擎报错。表现为切块状态标签显示失败，控制台打印引擎的标准错误输出。常见原因是文件不是 UTF-8、文件为空、文件结构异常导致解析失败。处理方式是按提示修正源文件，或先跳过该文件处理其它文档。

第三类是子进程超时。单文件切块超过三百秒会被强制结束，产物不发布，临时目录保留以便排查。法规文本正常在数秒内完成，触达超时通常意味着文件异常庞大或引擎陷入循环，处理方式是拆分源文件后重试。

10.7 前端与缓存

页面白屏或样式错乱时，第一步是按 Ctrl+F5 强制刷新。静态文件请求带版本查询串，正常情况下不会命中旧缓存，但在开发期修改了文件而版本号未变时，浏览器可能仍用旧文件。第二步是看控制台是否有红色异常：接口返回 503 说明数据目录不可用，返回 404 说明静态文件缺失，返回 500 说明服务端有未捕获异常。

界面在本地存储中只保存一个偏好（范围条是否显示），不保存业务数据，因此清除浏览器数据不会丢失任何编辑。若怀疑状态错乱（例如显示了不属于当前文档的卡片），刷新页面即可，因为页面的数据源始终是接口返回的完整载荷，不存在只能靠增量更新维持的状态。

10.8 并发与一致性边界

本软件按单用户设计。同一份文档被两个浏览器标签页同时打开时，两边的编辑各自保存在内存与草稿文件中，后保存的一方会覆盖先保存的一方，草稿文件名相同，因此不会产生两份草稿。提交时后提交者会因基线摘要不匹配而被拒绝（先提交者已经改变了正式文件），这一机制意外地提供了跨标签页的基本保护。

编辑过程中若后台正在切块，可能出现「切块完成触发重载，覆盖了当前编辑」的情况。防止的措施是：切块任务完成后的重载只更新基线，不丢弃内存中的命令栈；若基线摘要变化，草稿被标记为过期，界面提示使用者处理。更稳妥的做法是编辑前关闭自动切块开关，这一点在使用说明中有明确建议。

多实例的情况也需要注意。程序在启动时探测端口，若已有实例就不重复启动，因此正常操作不会出现两个进程同时写同一份草稿。若使用者强制用不同端口启动第二个实例，两个实例会各自持有内存状态，草稿文件互相覆盖，这是不被支持的用法。

10.9 边界条件汇总

下表把上述边界与另外几项较少见的情况汇总，便于排查时快速对照。

场景	现象	处理方式
源文件被改但未重切	左栏显示「过期」，对齐可能失真	提交或放弃草稿后按顺序重切
产物目录被删	文件从清单消失	重新切块或从备份恢复
备份超过三份	旧备份被自动删除	需要长期留存的备份请手工另存
删除最后一个分片	该组消失	用撤销恢复，或从备份复制
编号出现空洞	合并后编号不连续	属正常现象，不影响检索
表格行新增后缺口标记仍在	未切片行仍显示斜纹	已知限制，精确到行需引擎提供行号
表格被切块器标为段落	卡片出现「表格·切块器标为段落」徽章	用类型下拉改为表格，或按表格行重切
浏览器缩放非 100%	表格行定位可能偏移	恢复 100% 缩放后重试
中文路径	启动脚本与打包产物已处理	若仍异常，改用纯英文路径排查
磁盘写满	保存草稿失败并提示	清理磁盘后重试，草稿未写入不会损坏基线

表格中「表格行新增后缺口标记仍在」这一条需要额外说明，它属于契约层限制而不是实现缺陷当前契约只有块级的行区间，没有行级来源，因此从表格缺口新增的切片只能记录整张表所占的那一行，界面无法据此判断「这一行已经被新切片覆盖」，缺口标记会继续显示。要根治需要切块引擎在侧车中提供每块覆盖的工作行号，这属于后续阶段的扩展点。

第 11 章 测试与验收

11.1 测试策略

测试分四层进行，覆盖的粒度从函数到整机。第一层是单元测试，针对不依赖 HTTP 与磁盘状态的纯函数，主要是编辑命令、对齐、自动分片、编号与差异统计。第二层是接口测试，用真实的请求对象驱动路由层，覆盖状态码、错误文案与请求体校验，不启动真实端口。第三层是组件测试，针对草稿事务、清单扫描、配置读写、切块调度，用临时目录模拟真实文件系统。第四层是真机回归，在浏览器中按操作单逐项手测，记录实际数值。四层之外还有一项全语料统计，用真实法规语料跑对齐与切块，得到总体覆盖率。

这样分层的原因是各层的失败成本不同。单元测试最快，能在几秒内发现算法回归；接口测试能发现错误码与文案的退化；组件测试能发现文件系统边界上的问题；真机回归能发现浏览器渲染与交互的问题，但最慢。日常开发以单元测试为主，提交前跑一次全量单元测试与真机重点项，发布前跑打包冒烟。

11.2 单元测试分布

测试目录下有十四个测试模块，共二百一十七个测试函数，全部通过时耗时约二点八秒。分布如下表。表中数量为测试函数数，不是断言数；单个测试函数内通常有多个断言。

测试模块	用例数	覆盖内容
test_ops	48	八种命令、编号重排、分片重排、条号推断、差异统计
test_api	45	全部路由、状态码、错误文案、上传校验、静态白名单
test_align	22	三种对齐层、状态判定、回卷、表格行、区间定位
test_autopart	19	文本装箱、硬切、尾段重排、表格按行拆、混合块拆
test_config	19	默认值、校验、合并、保存与重载、路径规范化
test_manifest	12	旧清单读取、字段归一、统计
test_dir_manifest	10	目录扫描、键与分组、源缺失、两种命名兼容
test_drafts	10	草稿命名、保存、丢弃、备份轮转、提交事务、过期检测
test_chunker	9	引擎解析、待切扫描、命令拼装、任务触发与去重
test_paths	7	运行形态、应用根、默认配置、目录骨架
test_server	6	请求分发、请求体上限、响应头、实例探测
test_launcher	4	冻结态参数转发
test_real_corpus	4	真实语料上的对齐与契约字段完整性
test_studio_autopart	2	门面层自动分片与命令重放的组合行为

从分布可以看出投入的重点：编辑命令与路由合计九十三例，接近半数。这两块是使用者直接触达的部分，出错的影响也最直接。对齐与自动分片合计四十一例，覆盖的是一旦出错就很难人工发现

的算法。测试中没有对前端脚本写自动化用例，前端的验证依靠真机回归，这是当前阶段的取舍：前端逻辑以渲染为主，用无头浏览器测试的搭建成本高于收益。

11.3 关键测试点

编辑命令的测试覆盖了每一类字段变化。以拆分为例，用例包括：多行块按行切、单行块按字符切、切点越界报错、上半块保留编号、下半块取最大序号加一、两半分片字段重置、下半块条号显式指定、下半块条号自动识别、下半块条号继承、拆分后行区间重新定位、拆分后可撤销。合并的用例包括：相邻合并、参数顺序颠倒、非相邻报错、正文以换行连接、编号保留前块、条号从前块取、侧车从前块取、区间并集、任一块无区间时置空、集体合并保留首块、集体合并要求连续。

对齐的测试用小型构造语料，逐条断言状态与区间。用例包括：首行相等判为精确、首行被包含判为包含、面包屑剥离后命中、游标回卷命中、表格网格连续行命中、表格块回退到表题锚定、超长单行被多个块复用、相似度兜底命中、相似度不足判为未命中、窗口定位不越界。

接口的测试直接构造请求对象，断言状态码与返回体字段。用例包括：健康检查返回程序名、清单在数据源缺失时返回 503、未知键返回 404、缺少键返回 400、空请求体返回 400、非法 JSON 返回 400、请求体过大返回 413、上传文件名含路径分隔符被拒、扩展名不在白名单被拒、同名校验进入跳过列表、静态文件白名单外的路径返回 404。

草稿事务的测试用临时目录，覆盖正常提交、基线变化后拒绝提交、草稿缺失报错、备份轮转到三份、提交后草稿被删除、正式文件内容只含十五个字段。这一组用例是数据安全的关键，因此数量不多但每条都很具体。

11.4 真机回归

真机回归在浏览器中按操作单执行，逐项记录块数、编号与像素级结果。下表列出本版本验收时的主要项目与实测值。这些数值同时也是功能真实性的证据：每一项都对应一次真实操作后的界面读数。

回归项	操作	实测结果
拆分首块	消防法首块按行拆	块数 74 变 75，差异计数 加 1、改 1、删 0
集体合并	选中连续多块合并	差异计数 改 1、删 N，块数按选区减少
字符级拆分	单行块按字符拆	两半行区间相同，编号连续
表格行拆分	表格块按行区间拆	两块条号分别为「表 X 第 a-b 行」，网格各自完整
向上合并	与上一块合并	编号取上一块，正文顺序正确
向下合并	与下一块合并	编号保留当前块，区间并集
扩展文字块	从块首行拖过块尾	块数 74 变 73，目标块区间变为第 17 至 19 行
扩展表格块	表格行扩选	块数 185 变 183，条号写为「表 A.4 第 1-5 行」
并入缺口	从未切片处拖进下方块	块数 77 不变，区间由第 22 至 109 行变为第 21 至 109 行
表格缺口新增	未切片行右键新增	块数 836 变 837，条号预填「第二部分 裁量细则 表 1 第 215-215 行」
表格块右键新增	表格块选区新增	条号预填「附 录 C 表 1」
多选删除	选中两块删除	块数 836 变 834，一次撤销恢复为 836
表格多行选择	拖动选中三行	原生文字选区长度为 0，状态栏显示已选 3 行
表格行定位	点击第 120 行与第 300 行	分别定位到编号 0239 与 0373 的切片
滚动位置保持	在 3600 像素处删除第 700 行切片	恢复后位移 0 像素（另一次为 2 像素）
自动分片开启	阈值 200、最小 50	块数 185 变 442，差异 加 314、改 20、删 57
自动分片关闭	阈值改回默认	块数 442 变 185
草稿保存	编辑后等待两秒	徽章变为「草稿已保存」，草稿目录出现文件
提交	点提交	备份目录出现带时间戳文件，正式文件行数变化
放弃	点放弃并确认	卡片回到基线，草稿文件消失

表格行的多行选择单独做过一轮断言测试，共二十一项，覆盖单击选片、拖动跨行、Shift 扩展、拖到边缘自动滚动、Esc 清除、右键菜单项随选区变化、行高异常时仍准确等情况，全部通过。这一轮测试的背景是早期实现依赖浏览器原生文字选区，在行高很大时边界会落在单元格内部，导致选错行，改为自维护选区后问题消失。

11.5 全语料对齐统计

对齐质量用全量语料统计，统计口径是「切片是否有非空行区间」。下表列出主要文档的统计结果，末行为全量汇总。

文档	切片数	可定位	主要状态
消防法	74	74	精确与包含
GB 50016	531	531	包含 466、表格行 64、精确 1
GB 15499	294	294	包含与表格行
危险化学品目录	469	468	表格行 415、精确 48、包含 5
山东自由裁量基准	836	836	表格行 684、精确 146、包含 6
地方标准 2464	104	104	包含 84、精确 17、表格行 3
地方标准 2972	185	185	包含 79、表格行 71、精确 35
全部法规语料	13 764	13 671	包含 8 720、精确 716、表格行 4 206、表题 29、未命中 93
本地示例数据	1 046	1 045	未命中 1

全量定位率为百分之九十九点三二，未命中九十三条。未命中的分布与第 10 章的判断一致，集中在超长单行表与复杂合并单元格两类。这一结果在开发过程中有过两次提升：第一次由百分之九十六点三提升到百分之九十九点二八（三项改进之前的基础版本），第二次到百分之九十九点三二。提升的过程与具体改动见第 7 章。

11.6 打包冒烟

打包产物在发布前需通过十一项冒烟检查，全部在安装后的程序上执行，而不是在源码目录。检查项包括：双击可执行文件后页面能列出文件；设置对话框能打开且数据目录正确；把 Markdown 丢进源目录后三秒内自动切块；点文件后三栏联动正常；编辑后保存草稿、提交、备份目录出现文件；通过添加按钮导入并自动切块；关闭窗口后服务停止、重开可用；静默安装后程序存在且开始菜单有入口；以指定端口与不打开浏览器方式启动后，健康检查返回正常且引擎版本正确、数据目录位于安装目录；已安装版本丢入文件后自动切块成功；静默卸载后程序与快捷方式被删除，而配置与数据目录保留。

这十一项中，与路径有关的三项（数据目录位置、引擎目录、配置回落）最容易出问题，因为打包后的运行形态与源码运行不同。检查方式是在安装目录与用户目录各放一份配置，观察程序选择哪一份，以及设置面板显示的实际路径是否与选择一致。

11.7 验收口径

功能验收以操作单为准：第 9 章的每一步都能在真机上完成，且结果与界面反馈一致；第 8 章列出的每个界面元素都能找到并生效；第 5 章的每个接口都能返回预期结构。数据验收以契约与对齐统计为准：导出文件的每行恰好含十五个字段，派生字段可由正文重算得到相同值，全量语料的定位率不低于百分之九十九。性能验收以响应时间为准：打开单份文档不超过两秒，命中缓存后不超过零点二秒，编辑命令的响应在百毫秒级，千行文档滚动无可感知卡顿。稳定性验收以长时间使用为准：连续编辑数十次后撤销栈仍可用，草稿保存不产生重复文件，提交多次后备份目录保持三份。

不通过的判定标准也需明确：出现数据丢失（提交后块内容与界面不一致）、出现基线被破坏（提交前的校验被绕过）、出现不可恢复的错误（刷新后仍无法打开文档），任一情况即判为不通过必须修复后重新验收。样式偏差、文案措辞、非关键交互的手感差异不作为不通过的理由，记录后择期优化。

11.8 测试环境

单元测试与真机回归均在 Windows 11 64 位、Python 3.12.10 环境下进行，浏览器为 Chromium 内核的 Microsoft Edge。全语料统计使用本地法规语料目录，共九十七份 Markdown 文件；另有一份包含十个文档的示例数据目录用于日常开发。打包冒烟在打包机上执行，打包机装有 Python 3.10 以上版本、PyInstaller 与 Inno Setup 6。

测试数据不进入版本库。单元测试所需的夹具在测试代码中构造，接口测试使用临时目录，真机回归使用示例数据目录的副本。这样做的目的是让测试可在任何一台干净的机器上复现，而不依赖某台机器上的特定语料。

第 12 章 版本与维护说明

12.1 阶段划分

软件的开发按能力成熟度分为六个阶段，共计 69 个 commit，编号从 P0 到 P5。分阶段的目的是让每一阶段都有可演示、可验收的产物，而不是等到全部功能完成才第一次跑起来。各阶段的内容与状态如下表。

阶段	内容	状态	时间
P0	骨架：数据读取、服务与路由、三栏静态页面	已完成	开发第一日
P1	只读可视化：行模型、三层对齐、三栏联动、状态徽章	已完成	2026-11-03
P2	编辑：八种命令、撤销重做、差异计数、快捷键	已完成	2026-11-03
P3	持久化：草稿、提交、放弃、导出、备份轮转、导入、自动切块	已完成	2026-11-05
P4	参数化重切：分组参数覆盖、新旧结果对比	未实施	后续
P5	行级来源：切块引擎在侧车中提供工作行号	未实施	后续

P0 阶段用了半日，产出的是一个能列出文件、能打开页面、但右侧只有空卡片的骨架，主要验证了三件事：本地目录能被扫描成清单、源文件能被读成行、静态页面能通过接口取到数据。P1 阶段用一日，把对齐算法从零写到能在真实语料上达到九成以上的定位率，并完成三栏联动与状态徽章 P2 阶段用一日，把编辑动作从「只能看」推进到「能改」，重点是命令的原子性与撤销的正确性。P3 阶段用了约两日，把修改从内存推进到磁盘，引入了草稿、提交、备份三件套，并顺带补上了导入与自动切块。

12.2 P1 与 P2 的完成内容

P1 完成的内容包括：源文件行模型与超长行展开；三层对齐算法与七种状态；中栏的轻量格式化渲染、行号、范围条；右栏的卡片与元数据展示；双向联动、锁定、拖选反查；状态栏的对齐统计与源文件摘要校验；未命中与近似的视觉区分。这一阶段的验收标准是：任意文档在界面上都能看出每条切片对应哪几行，对不上的必须显示为未对齐。

P2 完成的内容包括：拆分的行级与字符级两种切法；合并、集体合并、扩展、并入、删除、批量删除、新增、修改八种命令；命令的校验与重放机制；撤销与重做；差异计数；快捷键；右键菜单拆分光标与气泡；表格行的着色与行选择。这一阶段的验收标准是：任何一次编辑都能用一次撤销还原，编辑后的块列表仍能通过契约校验。

12.3 P3 的完成内容

P3 完成的内容最多，可以归为五组。第一组是草稿与提交：草稿文件按基线摘要命名，自动保存带两秒防抖，提交时校验基线、备份、原子替换、删除草稿，放弃时删草稿回基线。第二组是导出按当前块列表下载 JSONL，含未提交的修改。第三组是导入：批量选择与拖放，文件名与编码校验同名跳过，大小限制，写临时文件再替换。第四组是自动切块：引擎解析、待切扫描、子进程调用、临时目录发布、轮询与进度报告、手动触发。第五组是自动分片与表格增强：超长块自动物化为真实分片、表格按行拆分、混合块按类型拆分、分片删除的三选一确认、分片泳道的双层表示。

这五组内容共同把软件从「编辑器」变成了「工作流工具」：语料可以进来，结果可以出去，中间的过程有草稿保护，异常的情况有明确提示。P3 结束时全量单元测试为二百一十七个用例，全部通过；全语料对齐率为百分之九十九点三二。

12.4 版本号与阶段标识

版本号有两处维护点。第一处是后端包内的版本常量与阶段常量，启动时打印到控制台，健康检查接口也会返回，界面顶栏的阶段标签读取的就是它。第二处是安装脚本中的应用版本号，决定安装包文件名与卸载项中显示的版本。两处需要同步修改，修改后重新打包。

版本号的取值规则为主版本、次版本、修订号三段，当前为 0.3.1，对应 P3 阶段。对外登记使用的版本号为 V1.0，与内部三段式版本是两套编号：内部编号用于开发迭代，登记编号用于软著材料。维护时应确保说明文档、安装包属性与登记信息三者的名称与版本一致，避免出现「文档写 V1.0、安装包写 0.3.1」这种不一致。

12.5 升级方式

切块引擎与工作台是两条独立的升级路径。引擎升级只需替换 vendor 目录下的引擎包，保持目录名不变，重启程序即可；新版本号会在控制台的引擎信息与设置面板中显示。替换时应同步更新 vendor 目录下的来源说明文件，记录新版本的来源、摘要与许可信息，以便日后追溯。引擎的许可是上游自有许可，对外分发前需要确认授权范围，这一点在打包文档中有提示。

工作台升级分两种。目录版升级是解压新版本覆盖旧目录，保留 data 目录与 config.json 不动；安装版升级是运行新安装包覆盖安装，同样不动数据。升级前建议先提交或放弃所有草稿，因为新版本若改动了草稿格式，旧草稿可能无法读取。当前版本的草稿格式为纯 JSONL，不含版本标记，格式变更时需要迁移脚本，这一点在未来的兼容性设计中需要补上。

12.6 备份与恢复

自动备份只在提交时发生，每次提交把旧的正式文件复制到备份目录，文件名带时间戳，保留最近三份。三份之外的旧备份被自动删除。这个策略覆盖了「连续改错三次」的场景，但不足以应对「一周后才发现改错」。需要长期留存的版本应手工另存，或在提交后把整个 data 目录复制到外部位置。

恢复的方式是手工的：从备份目录挑一个时间点，把文件复制回正式位置，覆盖当前文件，然后在界面上点刷新。恢复后当前会话的草稿会因基线变化而变为过期，需要放弃。恢复操作没有界面入口，这是刻意的——恢复属于低频高风险操作，做成按钮容易被误点，手工复制虽然笨拙但每一步都可见。

草稿不备份，只保留最新一份。放弃草稿是不可撤销的，如果使用者对当前修改没有把握，正确做法是先导出 JSONL 再放弃，而不是指望草稿能被找回。

12.7 数据维护

数据目录中有三类可清理的内容。第一类是草稿目录，长期使用后会积累已放弃或已提交后残留的文件（正常情况下提交与放弃都会删除草稿，残留多因异常中断），可以定期清空。第二类是备份目录，保留三份之外的不会自动出现，但若使用者手工放入过文件则需要自己管理。第三类是切块临时目录，切块失败时保留的临时产物可用于排查，确认无用后可删除。

迁移数据目录的方式有两种：把整个 data 目录复制到新位置并在设置中修改 data_root；或只复制 source 与 chunks，让程序重新生成草稿与备份目录。前者保留全部历史，后者更干净。data_root 支持绝对路径，因此可以把数据放在另一块磁盘或网络盘上，但网络盘的写入延迟会影响草稿保存的手感，不建议。

12.8 扩展点

软件预留了若干扩展点，后续开发可以在不破坏现有契约的前提下进行。第一是命令扩展：所有编辑动作都实现为「接收命令对象、返回新块列表」的函数，新增一种命令只需增加一个函数并在分发表中注册，撤销与差异统计自动获得支持。第二是展示字段扩展：接口返回的块对象已包含契约字段、公开侧车、行区间与状态，若要展示新的侧车键，只需把它加入公开白名单，不影响契约。第三是导出的扩展：导出当前走契约裁剪，若下游需要附带侧车，可以增加一个「导出含侧车」的选项而不改变默认行为。

第四是上传类型扩展：当前只接受三种文本扩展名，若要支持其它文本格式，改动集中在白名单与保存时的扩展名映射两处。第五是对齐层扩展：对齐函数按块类型分流，新增一类块只需增加一个分支，不影响已有分支。第六是前端渲染扩展：卡片的表格渲染与原文的表格渲染是两段独立代码，若要增加新的块类型可视化，可以只改其中一段。

P4 与 P5 是两个已经规划但未实施的扩展。P4 是在界面上提供分组参数覆盖与重切预览，让使用者可以调切块长度、表格模式等参数并对比新旧结果，落地位置是新增一个参数模块与一个对比弹层。P5 是让切块引擎在侧车中写入每块覆盖的工作行号，从根本上解决表格行级缺口与跨表块定位的问题，需要改动引擎而不是工作台，因此要走上游的提案流程。

12.9 已知限制

当前版本有五条已知限制，均已在正文相应位置说明，这里集中列出以便查阅。第一，单用户单实例：不支持多人同时编辑，同一份文档在两个标签页中编辑时后保存者覆盖先保存者，提交时的基线校验只能拦住「先后提交」而不能拦住「同时编辑」。第二，表格行级缺口：从表格缺口新增的切片只能记录整表行号，缺口标记不会消失。第三，无渲染视图：原文以源码行显示，不提供Markdown渲染后的阅读模式。第四，无参数化重切界面：切块参数只能在引擎侧调整，工作台只提供触发与展示。第五，静态响应未压缩：大数据量载荷在局域网外的远程访问下会偏慢，但本软件只服务本机，实际不受影响。

把限制写进说明书不是为了免责，而是为了让后续维护者知道哪些行为是设计选择、哪些是待补的缺口。例如「无渲染视图」不是遗漏，而是为了让行号与原文严格对应；「无参数化重切」不是不会做，而是当前的语料已经切好，重切的需求频次低于人工修订。

12.10 维护约定

后续维护应遵守四条约定。第一，先提案后动手：任何改变行为的修改，先写清现状、根因、改动文件、具体改法、影响面、验证方式与风险边界，经确认后再编码。这条约定在单人项目中同样有效，它能防止「边想边改」导致的返工。

第二，上游只读：不修改外部切块工程的代码与数据，不向其目录写入任何文件。需要引擎配合的改动通过提案沟通，而不是在工作台里打补丁。

第三，契约冻结：十五个字段不增不减不改名，需要携带的信息放侧车。任何看起来「只加一个字段」的改动都要拒绝，因为下游索引流程按固定字段解析，多一个字段可能被当成脏数据。

第四，测试先行：修改算法前先补一条能复现问题的用例，修改后确认该用例通过且全量用例不回归。二百一十七个用例跑完不到三秒，这个成本足以支撑每次修改都跑一遍。

本说明书描述的功能与行为以 2026 年 11 月 5 日的代码为准。若后续版本有改动，应同步更新本说明书对应章节，特别是第 5 章的接口清单、第 6 章的模块划分与第 11 章的测试统计，这三处最容易与代码脱节。